GEORG-AUGUST-UNIVERSITÄT
GÖTTINGEN

Deep Learning for Computer Vision

Recap: ML & DL Basics

Constantin Pape
Institut für Informatik, Uni Göttingen

https://user.informatik.uni-goettingen.de/~pape41/

cppape

Credit: lecture based on “Deep Learning” by A. Ecker
some of the slides based on Ian Goodfellow’s slides

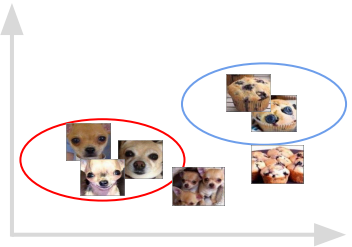
Agenda for today

- The basics of (supervised) Machine Learning
- The basics of Deep Learning architectures
  - Perceptron
  - Multi-layer perceptron
- Training neural networks
  - Loss functions
  - Backpropagation
  - Stochastic gradient descent
- Deep learning frameworks

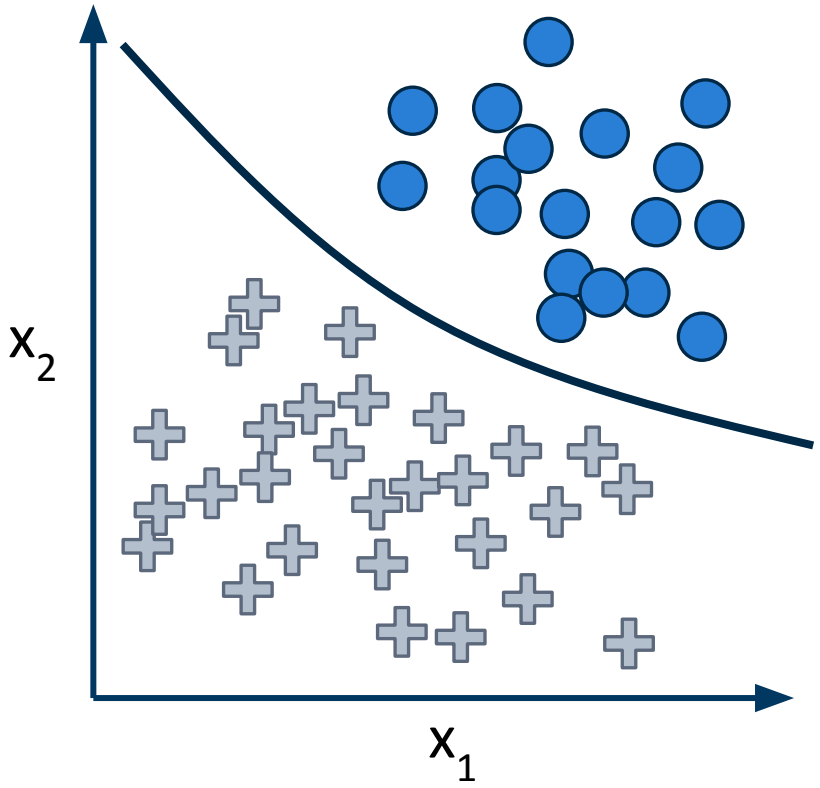
ML Basics

Supervised learning

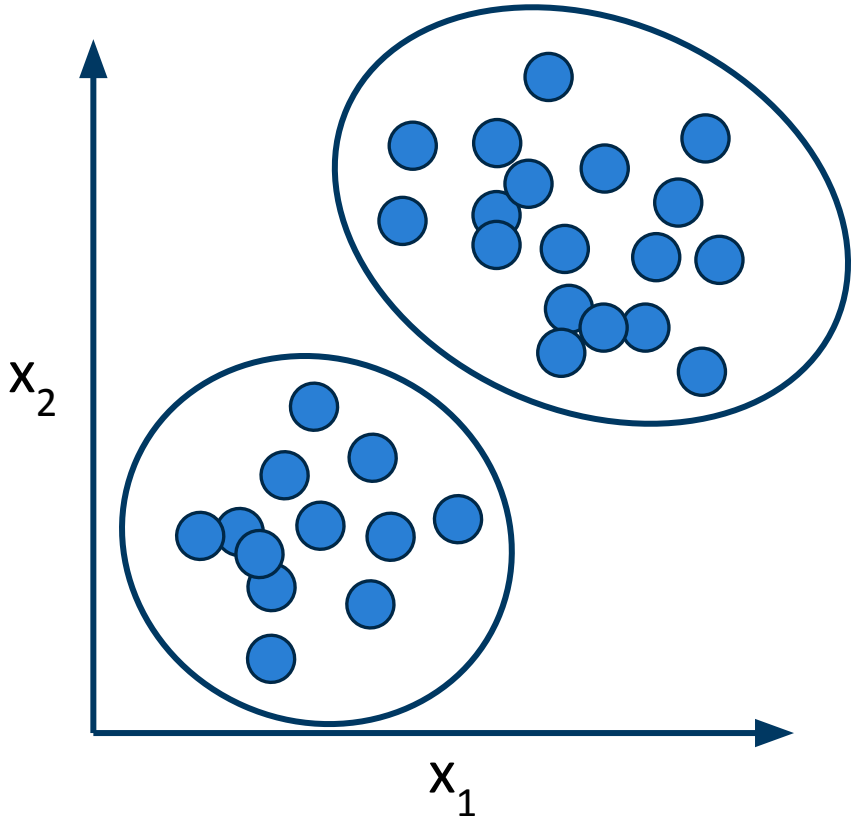
Learning paradigms

	Supervised Learning	Unsupervised Learning	Reinforcement Learning
	 Dog or Muffin?		
Description	Learn mapping from input to labeled output	Model underlying structure of dataset	Learn strategy to get reward when interacting with the environment
Feedback	Correct answer is given (labeled data)	No answer is given (unlabeled data)	Occasional reward, but no direct feedback on individual actions
Example	• Image classification • Machine translation	• Clustering • Dimensionality reduction	• Learning to play chess • Robot learning to perform a task

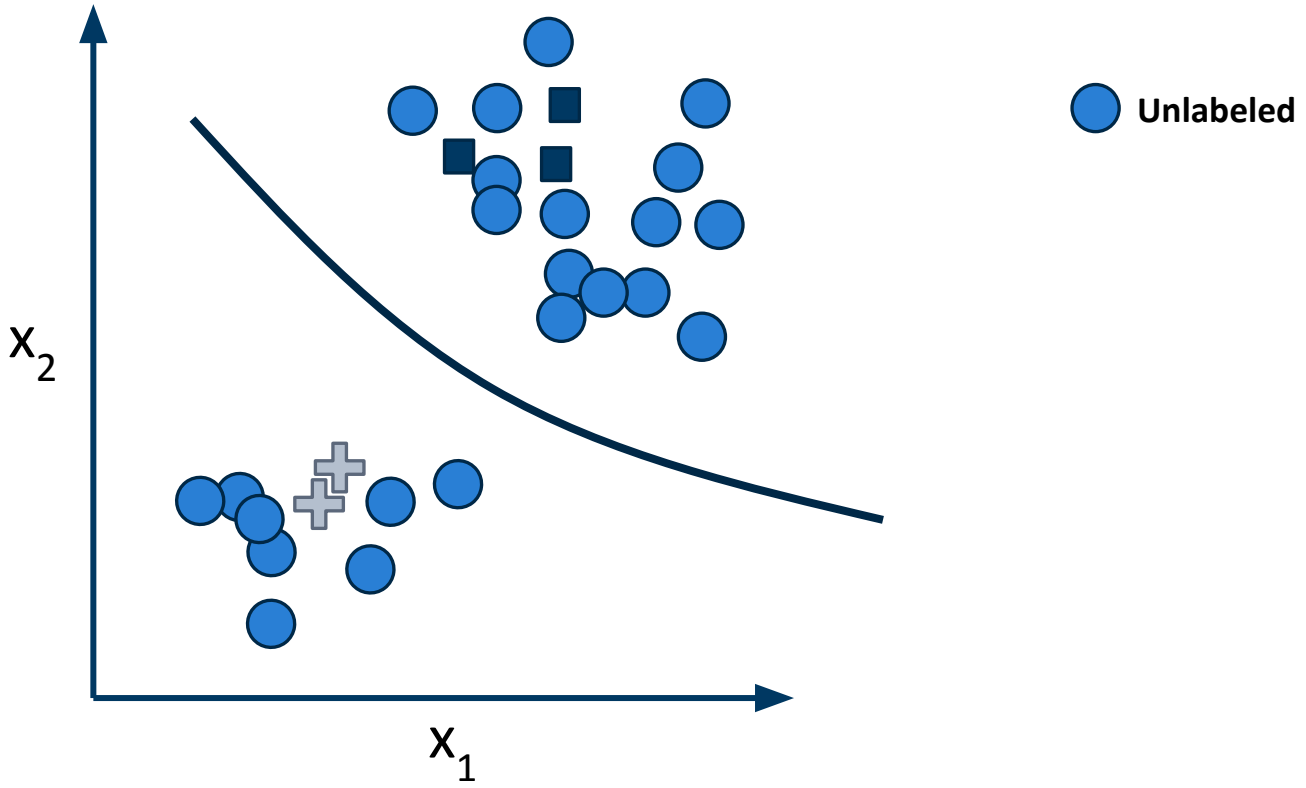
Supervised learning



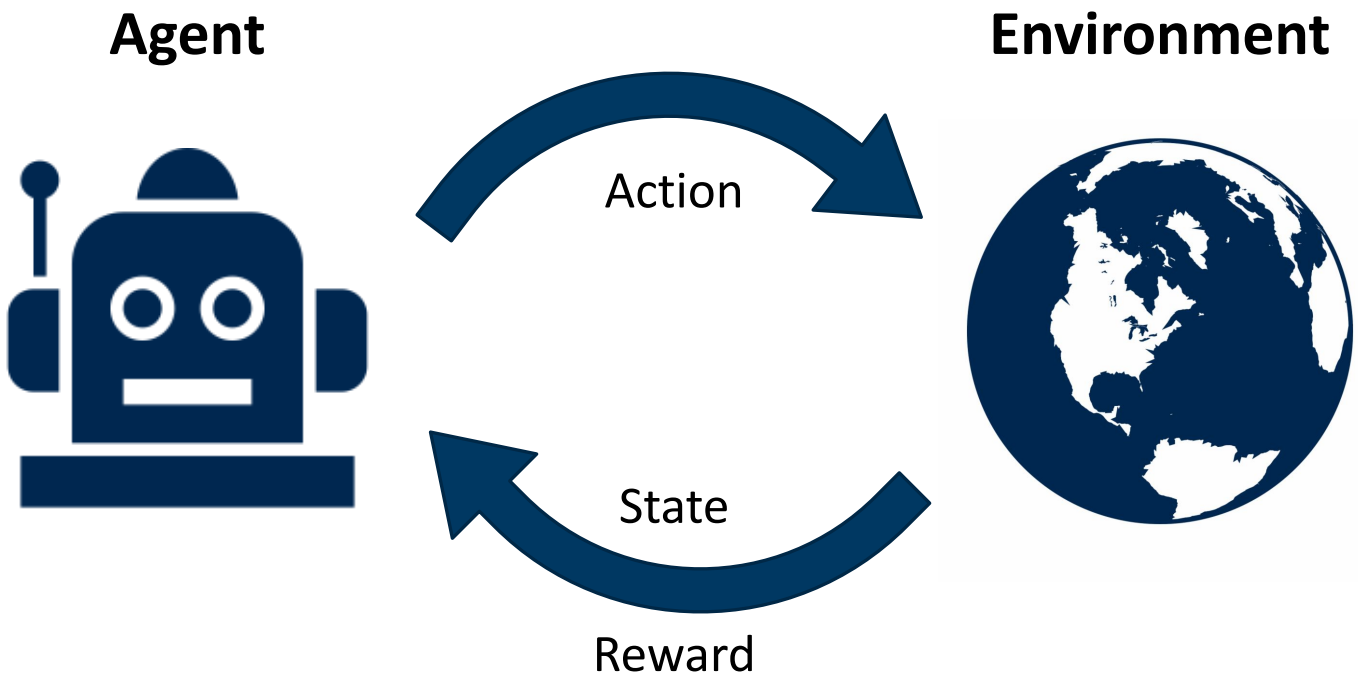
Unsupervised learning



Semi-supervised learning

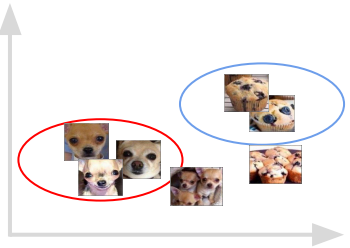


Reinforcement learning



Learning paradigms

Focus for today and also for first ¾ of the course

	Supervised Learning	Unsupervised Learning	Reinforcement Learning
	 Dog or Muffin?		
Description	Learn mapping from input to labeled output	Model underlying structure of dataset	Learn strategy to get reward when interacting with the environment
Feedback	Correct answer is given (labeled data)	No answer is given (unlabeled data)	Occasional reward, but no direct feedback on individual actions
Example	• Image classification • Machine translation	• Clustering • Dimensionality reduction	• Learning to play chess • Robot learning to perform a task

Supervised vs. unsupervised learning

Supervised learning

- Desired outputs are known and paired examples of inputs and outputs are given
- Tasks: classification or regression
  - and variants / combinations
- Algorithms:
  - Linear / logistic regression
  - Decision trees / random forests
  - Support vector machines
  - …

Unsupervised learning

- Paired examples are not available; desired outputs may be unknown
- Unsupervised learning tasks:
  - Dimensionality reduction: visualization
    - PCA, t-SNE, …
  - Clustering: group data in clusters without labels
    - k-means, hierarchical clustering, …
  - Feature learning: learn good presentation of the data
    - Auto-encoder, …

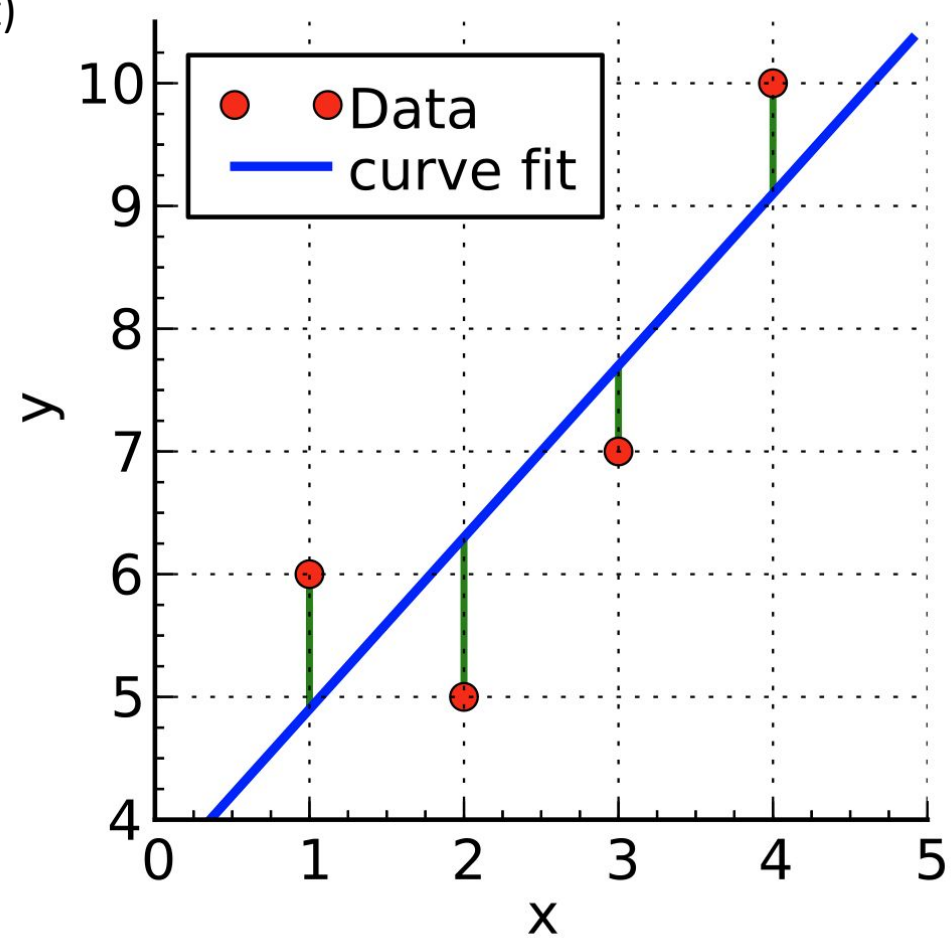
https://en.wikipedia.org/wiki/Linear_regression

Algorithms for supervised learning: Linear regression

Fit a line to the data points x (inputs/features) and y (output/target)

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^{\mathsf{T}} \boldsymbol{\beta} + \varepsilon_i, \qquad i = 1, \ldots, n,$$

- beta are the weights = parameters of the model
- fit to data by minimizing squared error (L2 loss)
  - via ordinary least squares (analytical solution)
  - or gradient descent (same idea as training neural nets!)



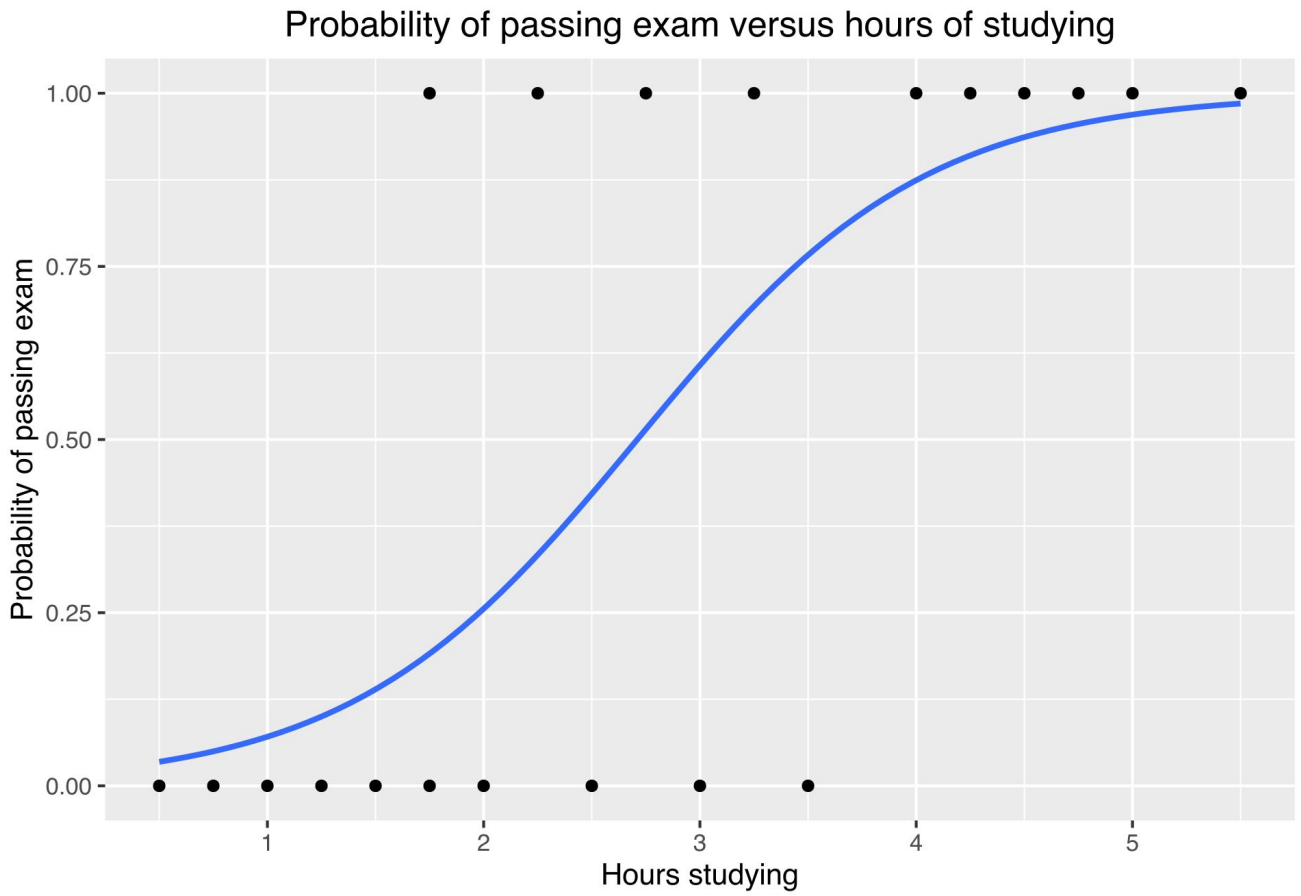
https://en.wikipedia.org/wiki/Logistic_regression

Algorithms for supervised learning: Logistic regression

Predict probability for a binary result with **linear** dependence on feature(s)

$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Binary classification problem



https://en.wikipedia.org/wiki/Logistic_regression

Algorithms for supervised learning: Logistic regression

Predict probability for a binary result with **linear** dependence on feature(s)

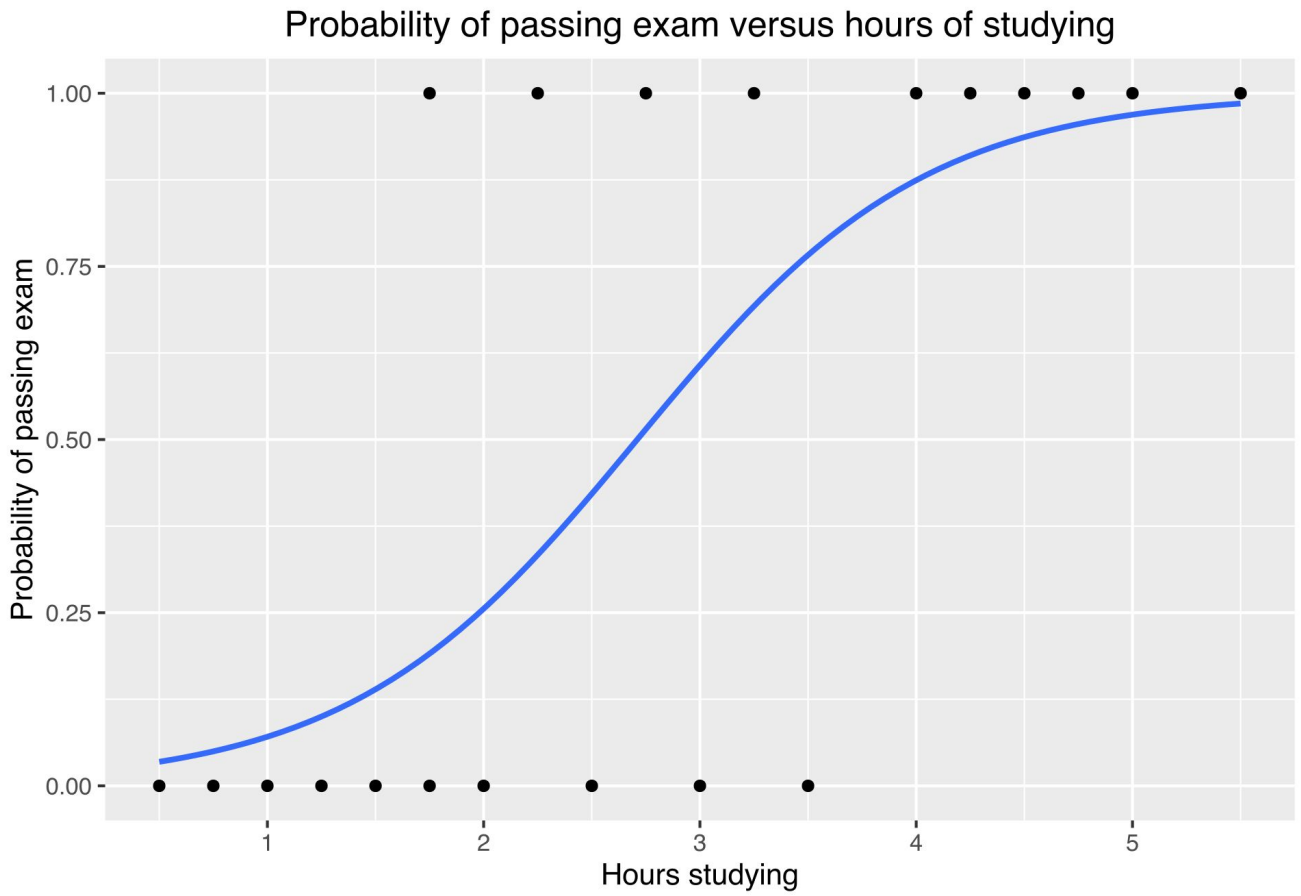
$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Binary classification problem

Generalized to multiple inputs:
"sigmoid of linear regression"

$$p(x) = \sigma(\mathbf{x_i^T}\ \beta)$$

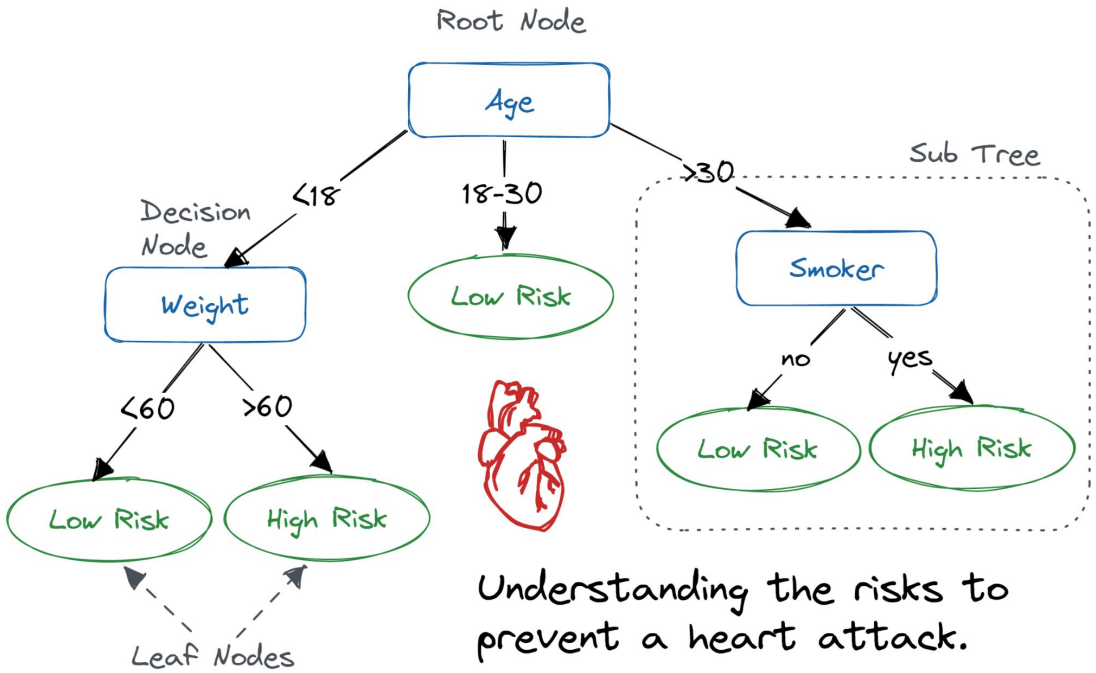
Non-linearity ->
no closed form solution ->
gradient descent



https://www.datacamp.com/tutorial/decision-tree-classification-python

More algorithms

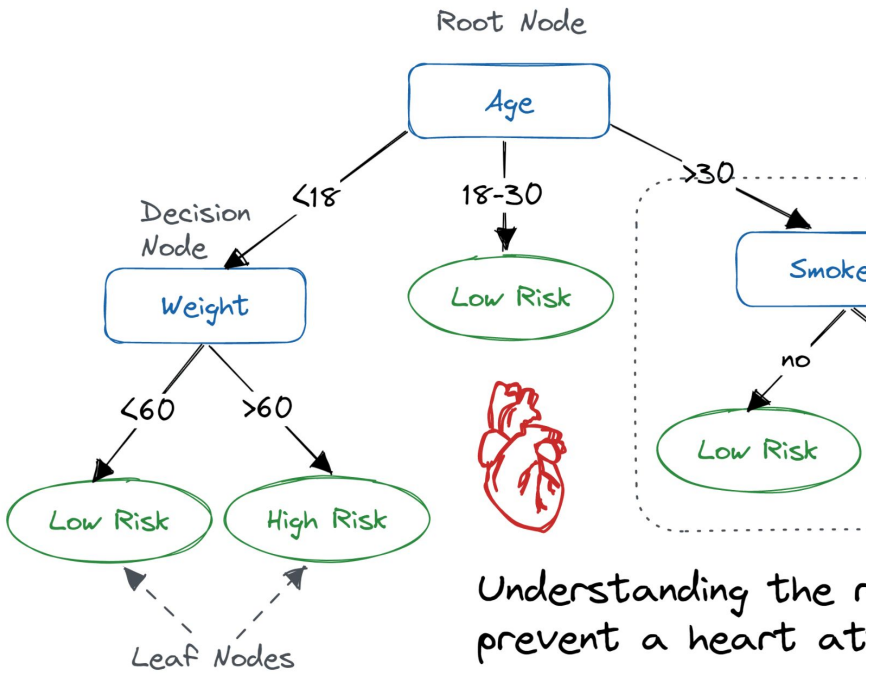
Decision Tree



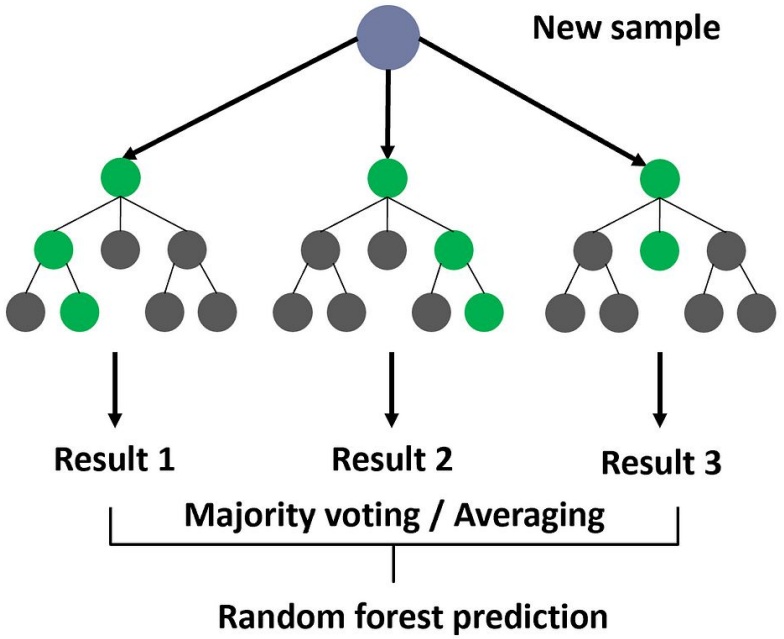
https://www.datacamp.com/tutorial/decision-tree-classification-python
https://medium.com/@roiyeho/random-forests-98892261dc49

More algorithms

Decision Tree



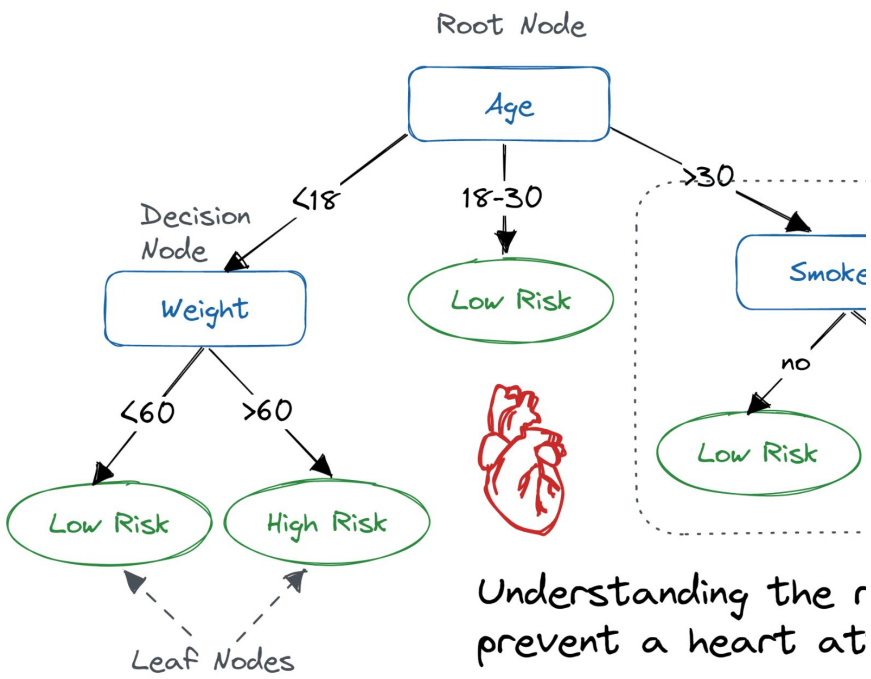
Random Forest



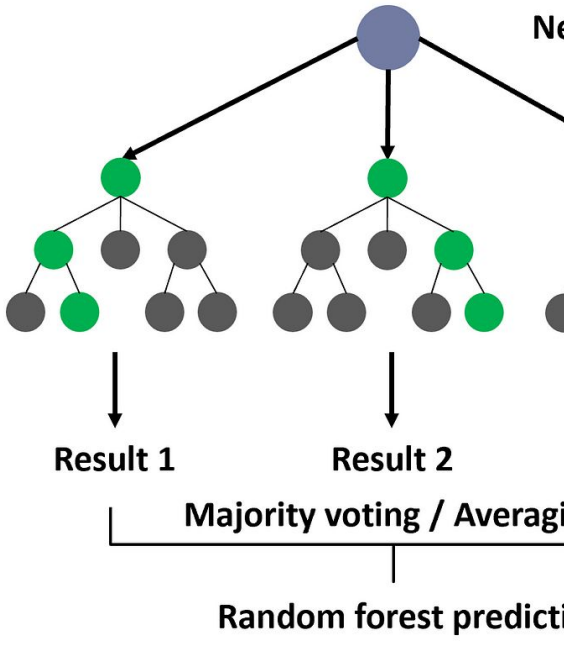
https://www.datacamp.com/tutorial/decision-tree-classification-python
https://medium.com/@roiyeho/random-forests-98892261dc49

More algorithms

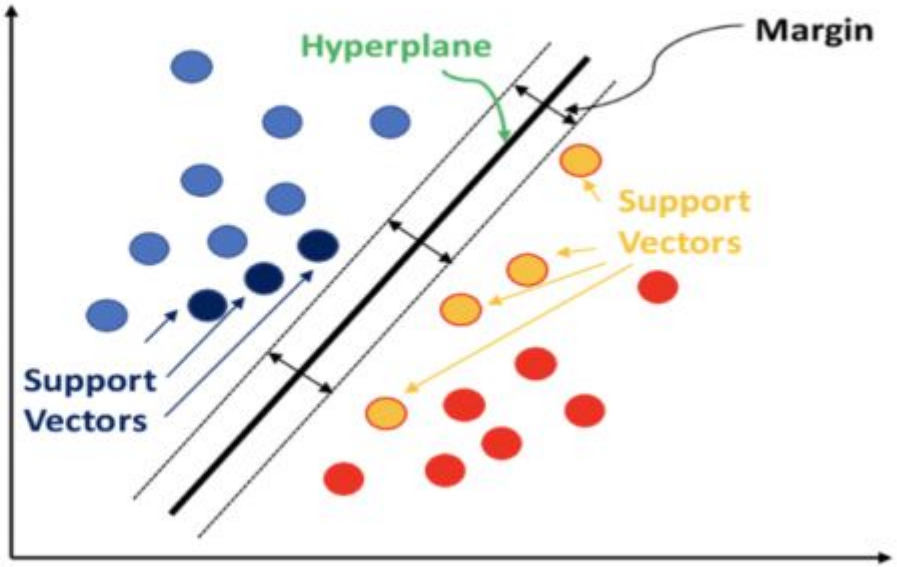
Decision Tree



Random Forest



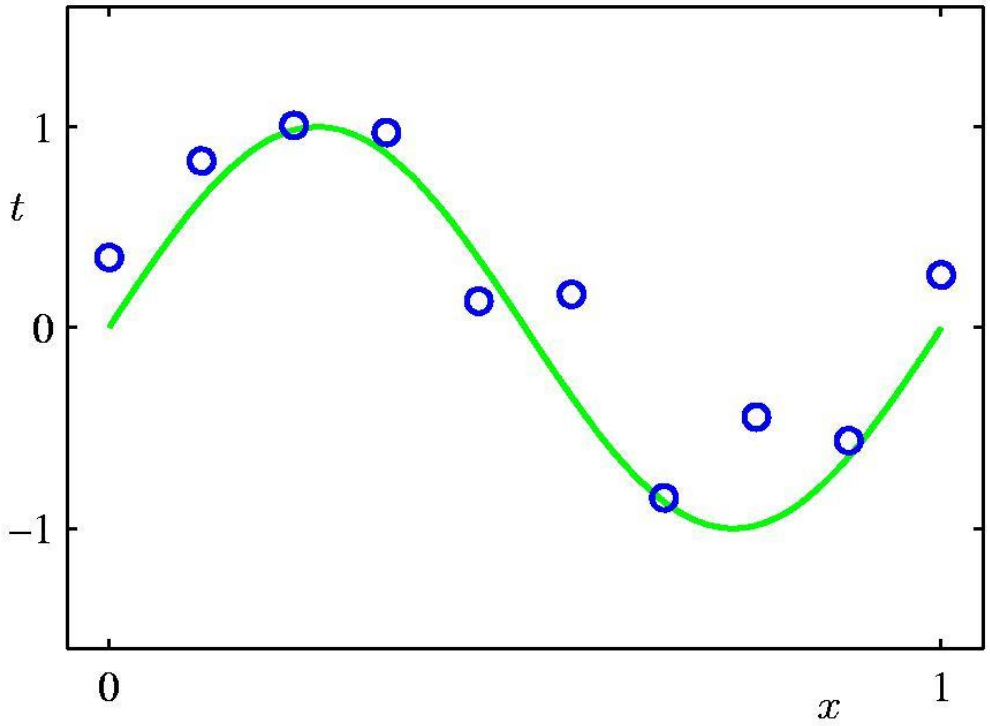
Support Vector Machines …



Case study

POLYNOMIAL CURVE FITTING

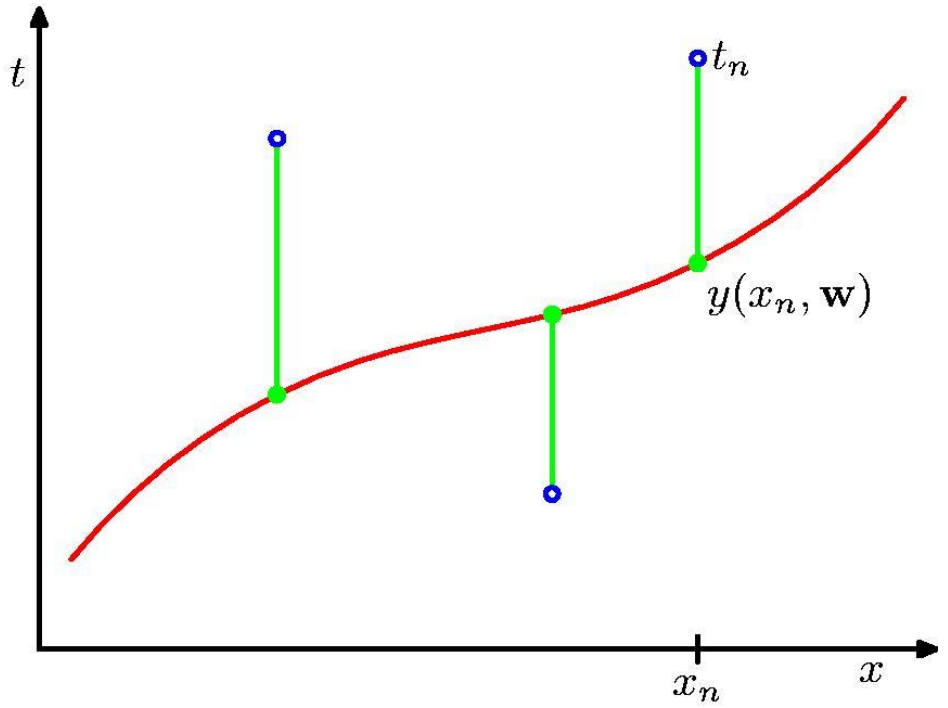
Polynomial Curve Fitting



$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

Bishop: Pattern Recognition and Machine Learning

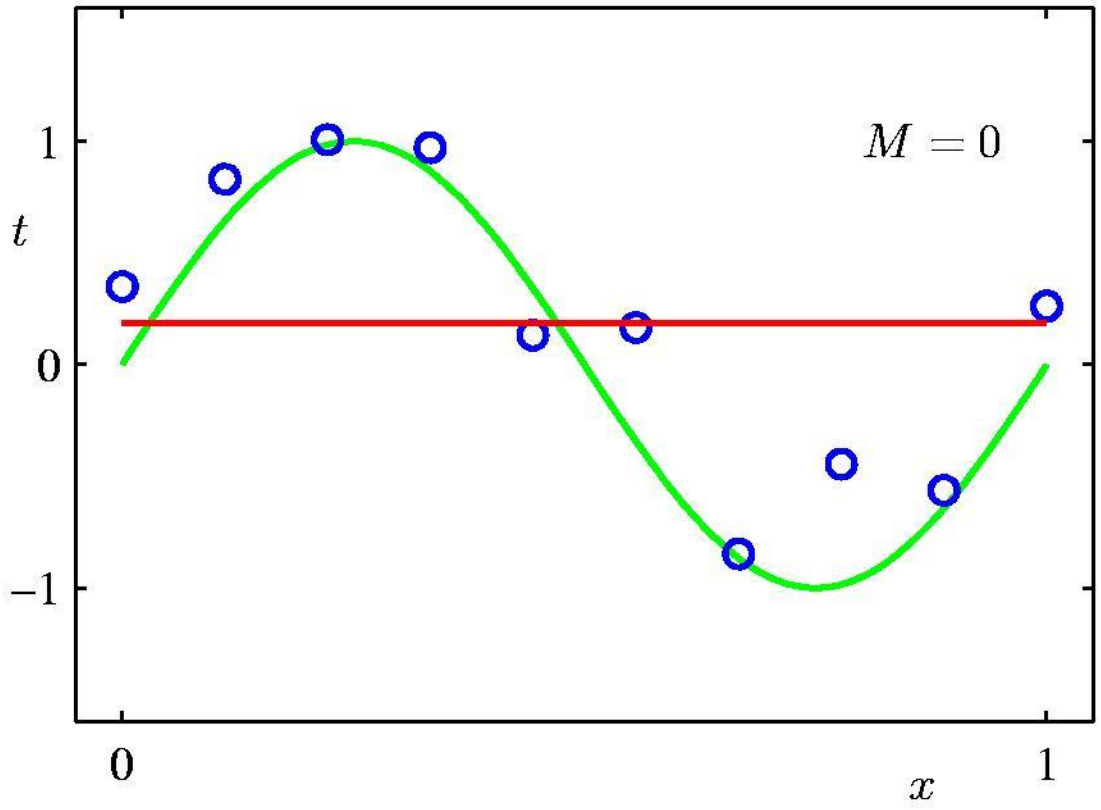
Sum-of-Squares Error Function



$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2$$

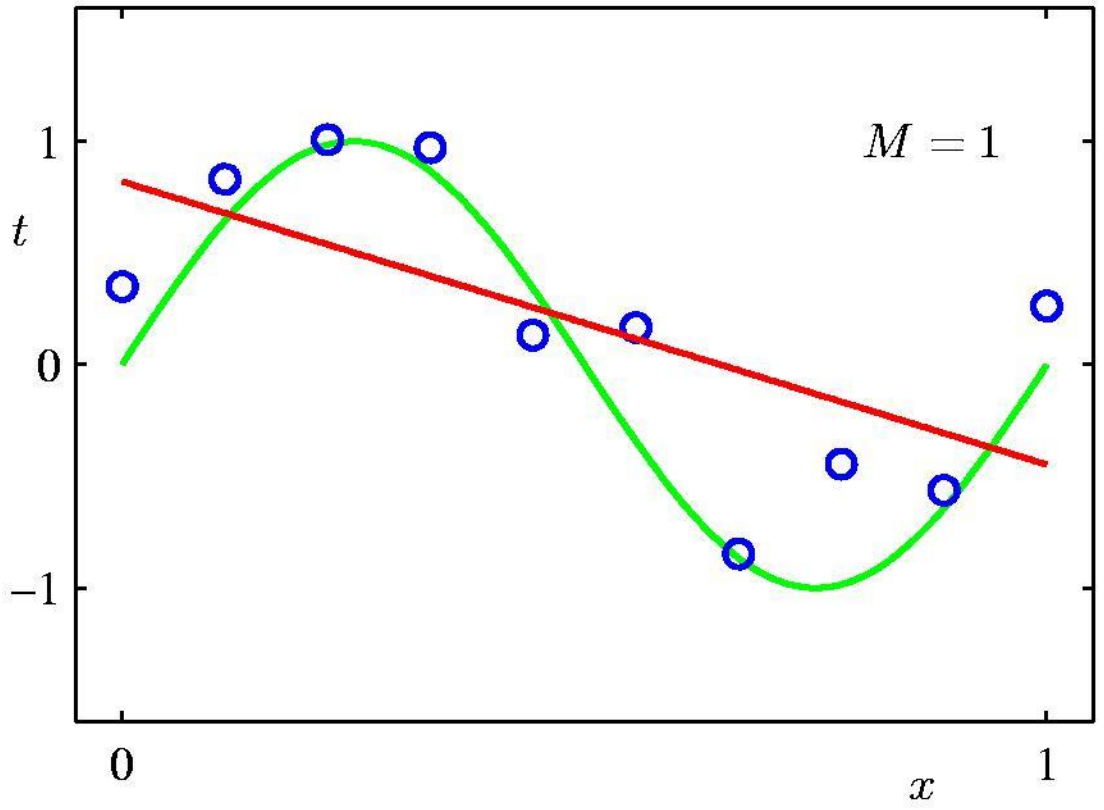
(Punishes far outliers more.)

0^{th} Order Polynomial



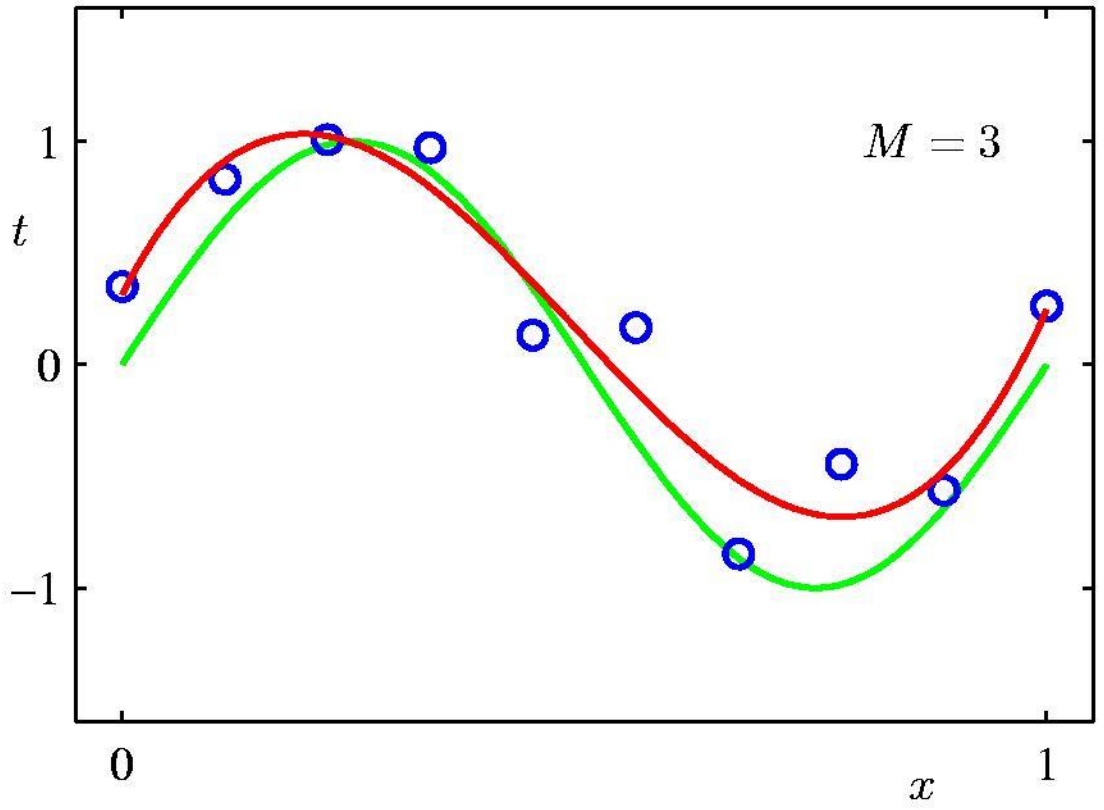
Bishop: Pattern Recognition and Machine Learning

1st Order Polynomial



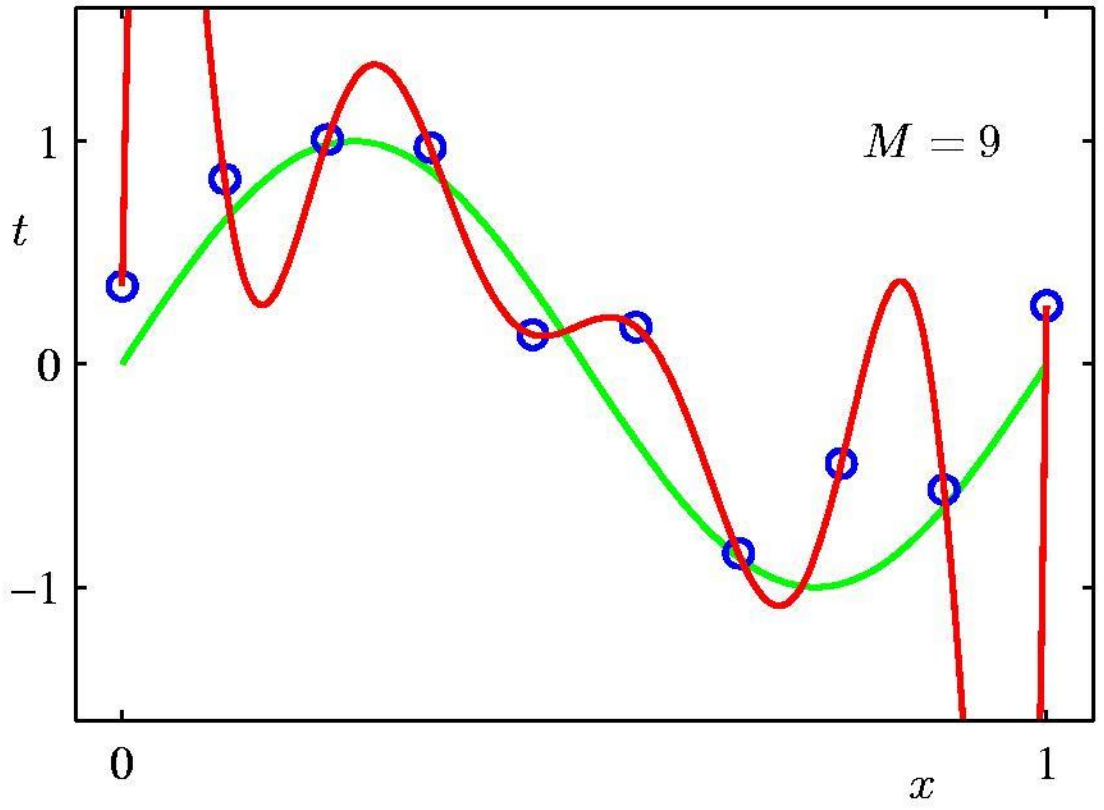
Bishop: Pattern Recognition and Machine Learning

3rd Order Polynomial



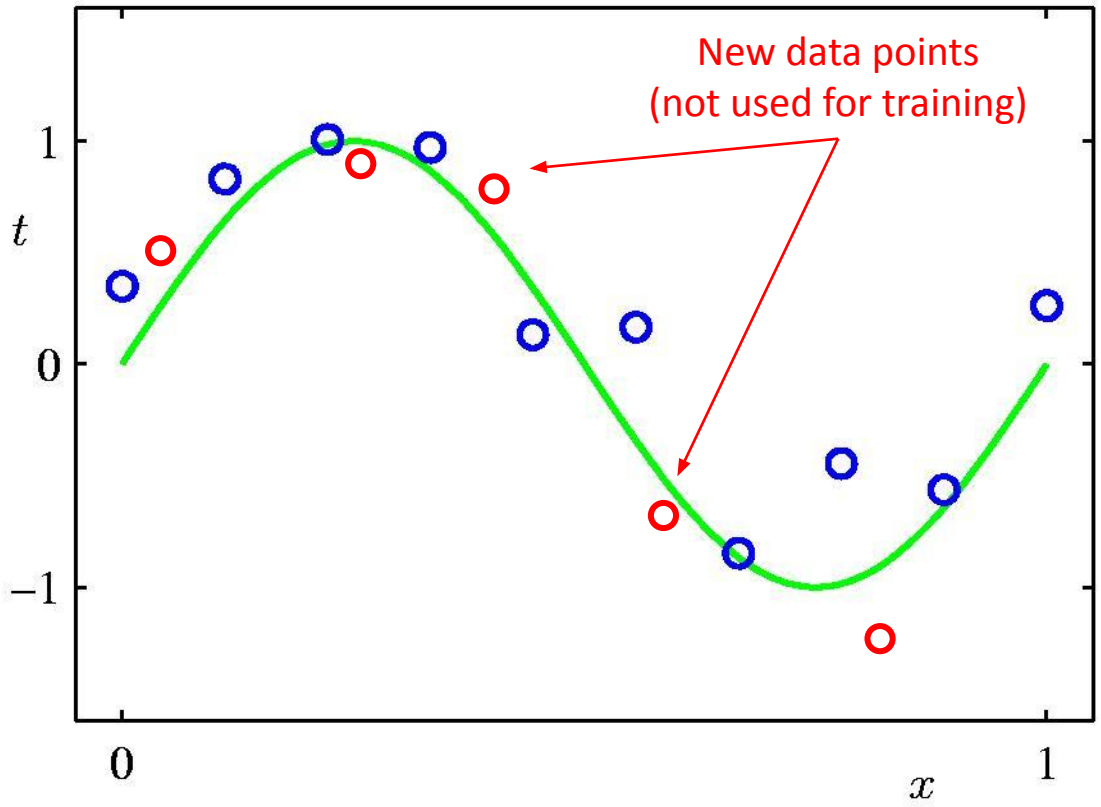
Bishop: Pattern Recognition and Machine Learning

9^{th} Order Polynomial



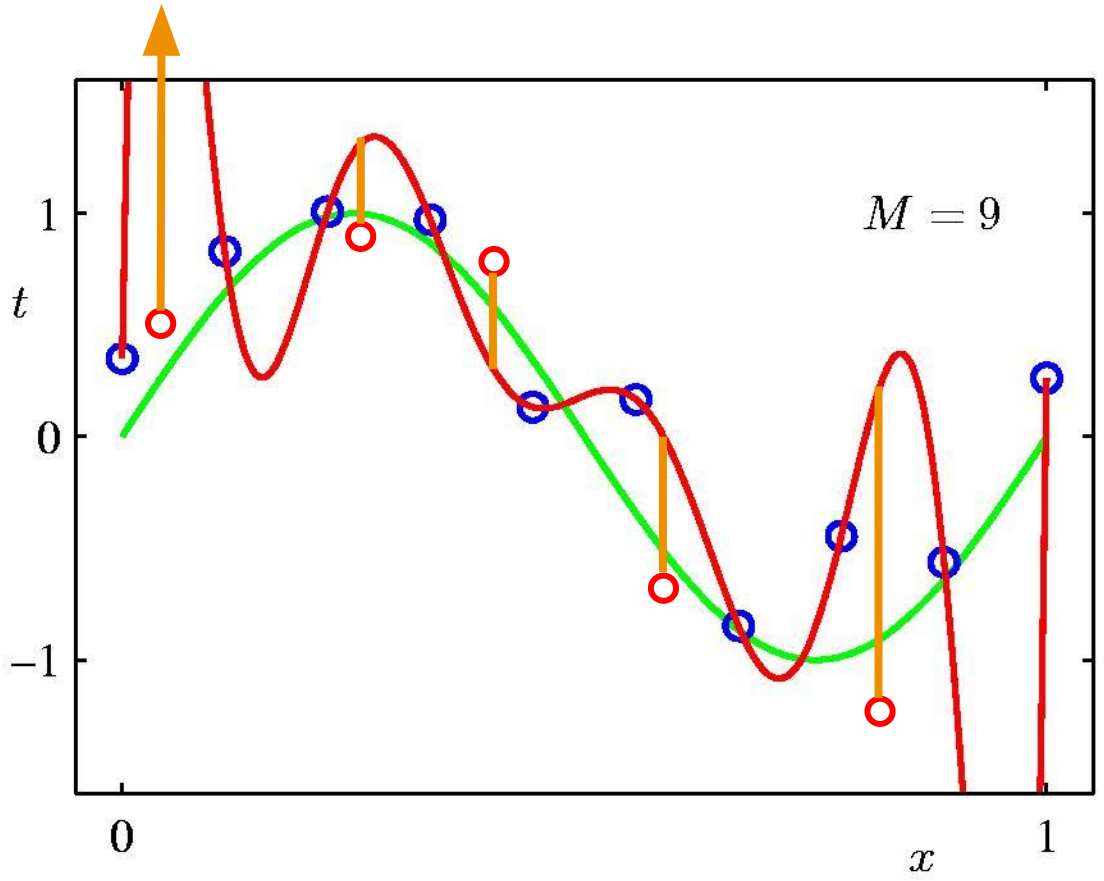
Bishop: Pattern Recognition and Machine Learning

Goal: Generalization to new data



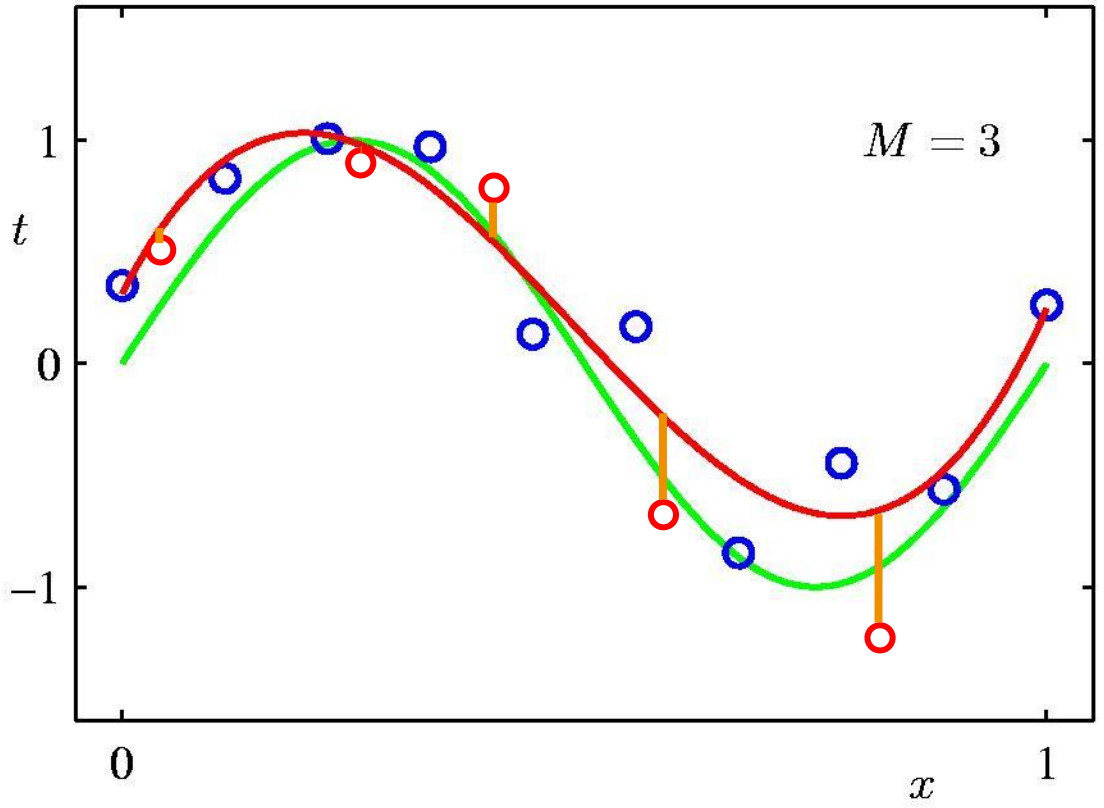
Bishop: Pattern Recognition and Machine Learning

9^{th} Order Polynomial



Training set: perfect fit
Test set: **large errors!**

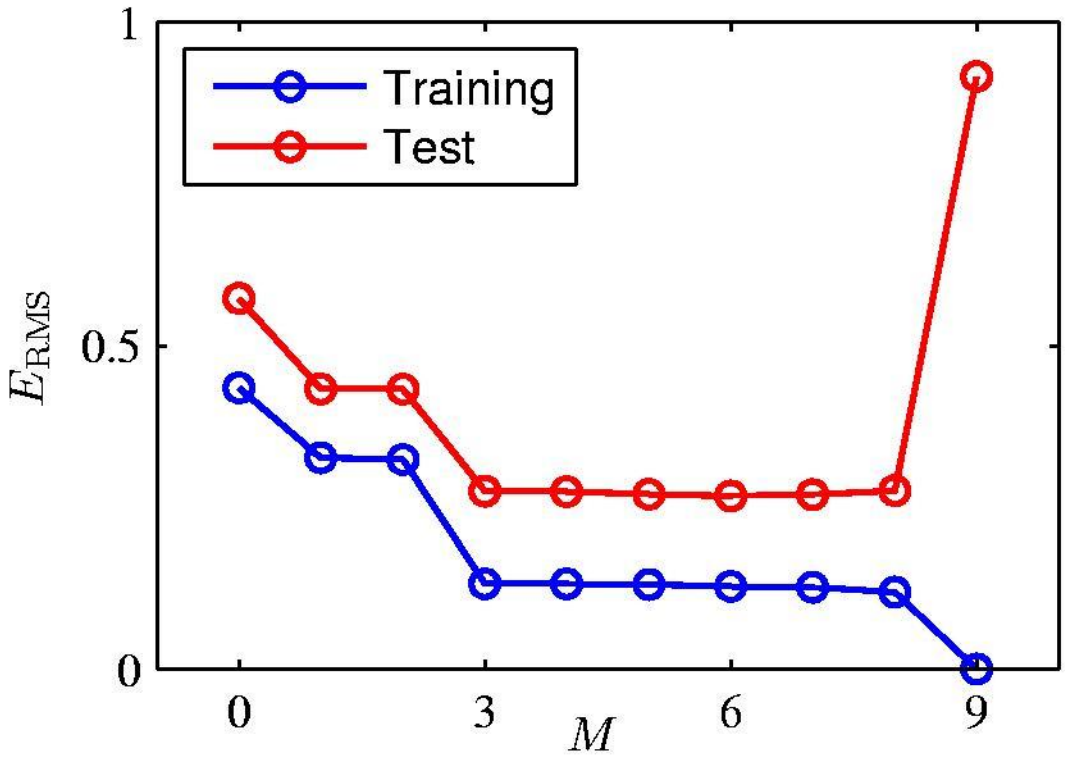
3rd Order Polynomial



Training set: small errors
Test set: **small errors**

Bishop: Pattern Recognition and Machine Learning

Overfitting

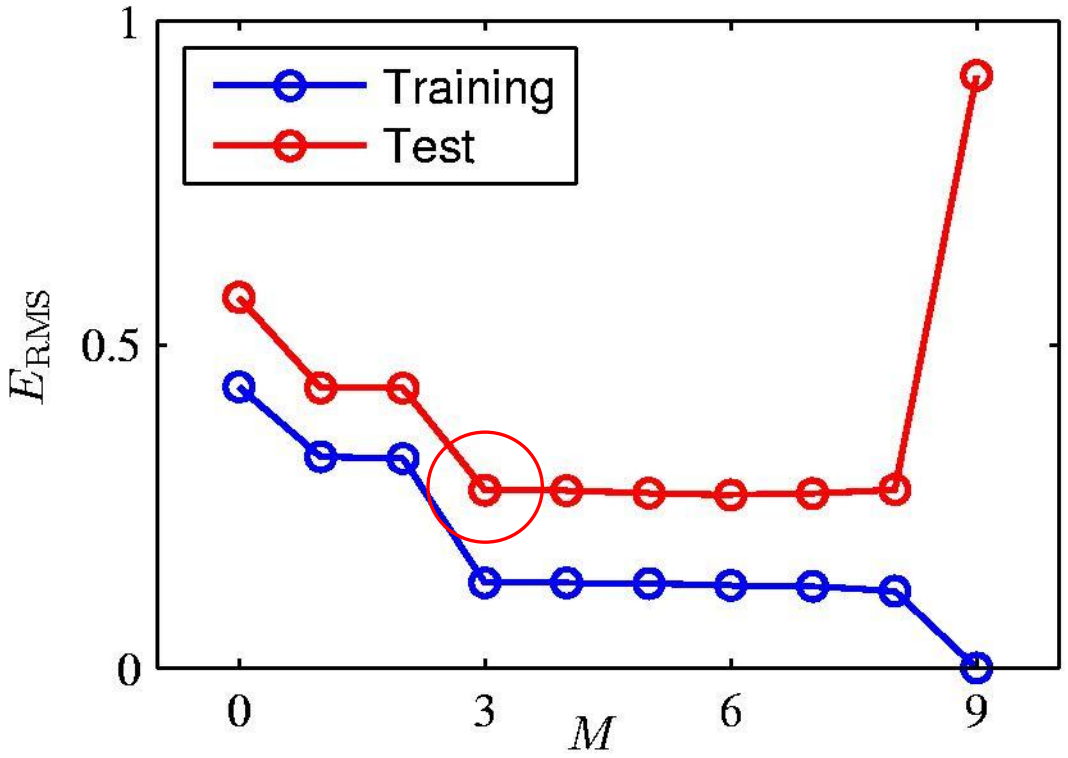


Root-Mean-Square (RMS) Error: $E_{\mathrm{RMS}} = \sqrt{2E(\mathbf{w}^{\star})/N}$

Polynomial Coefficients

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

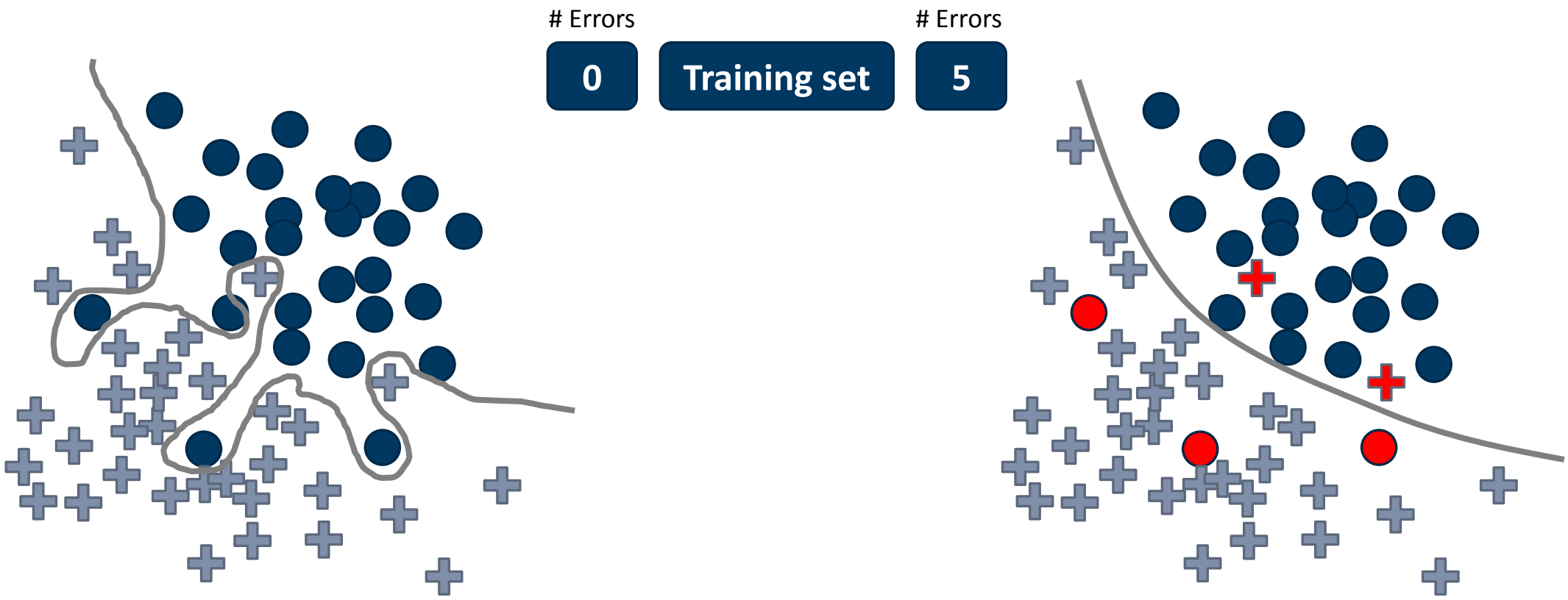
Occam’s Razor



William of Ockham: "Plurality must never be posited without necessity“. Choose the model at the kink.
(Or consider a different class of hypothesis/models.)

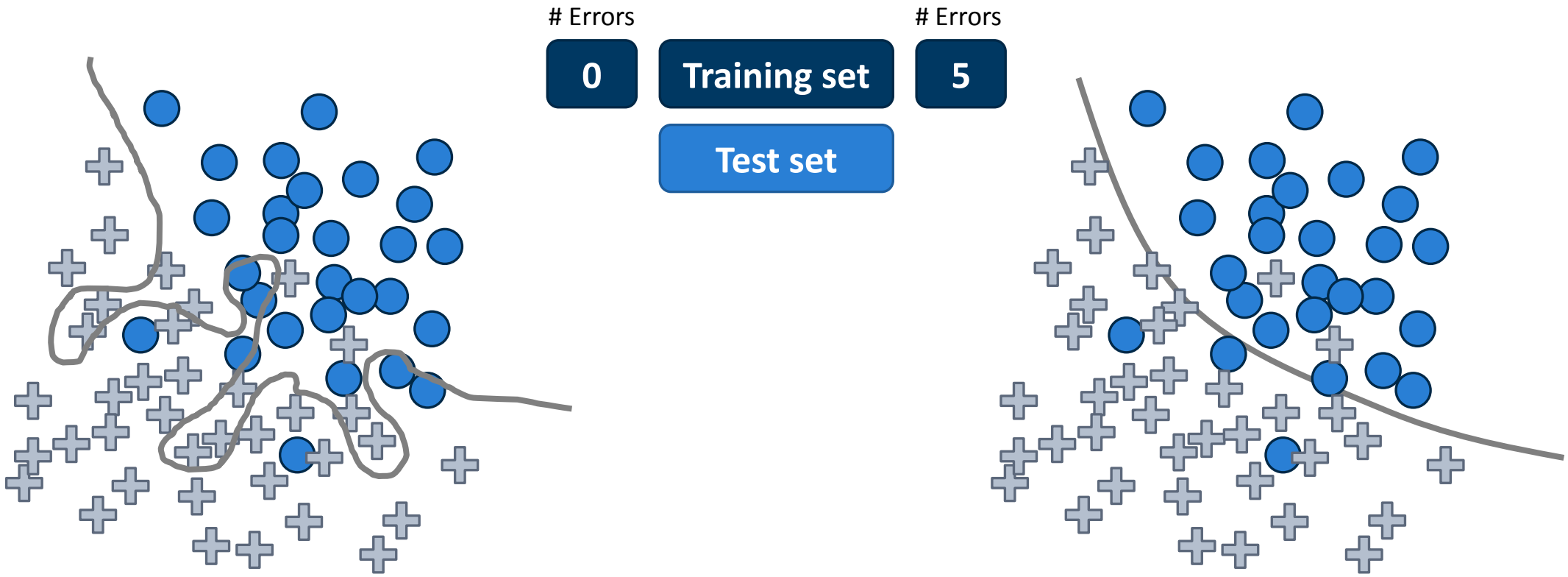
Over-fitting

Fitting a complex function can minimize the training error.



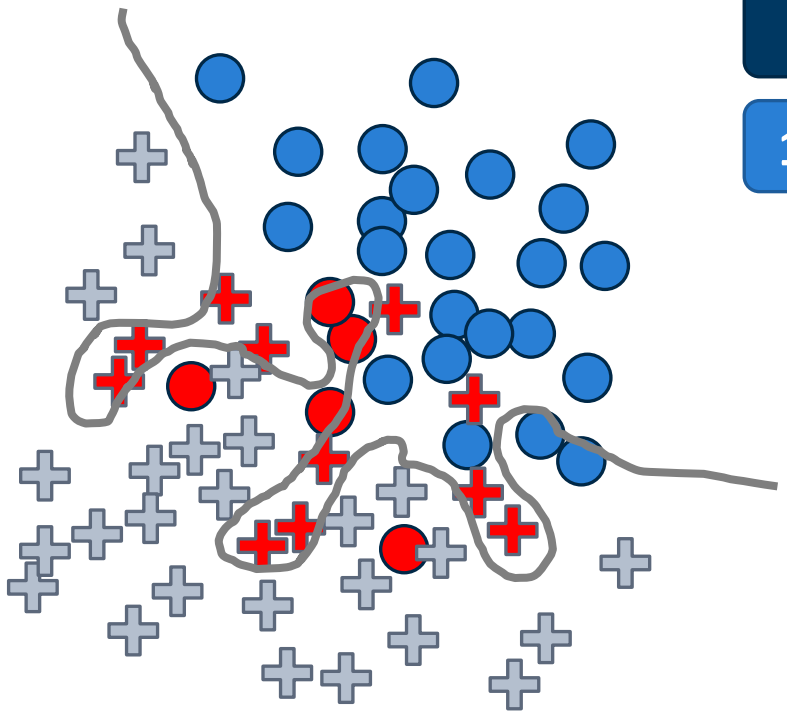
Over-fitting

What happens when we deploy our classifier and test it on new data?

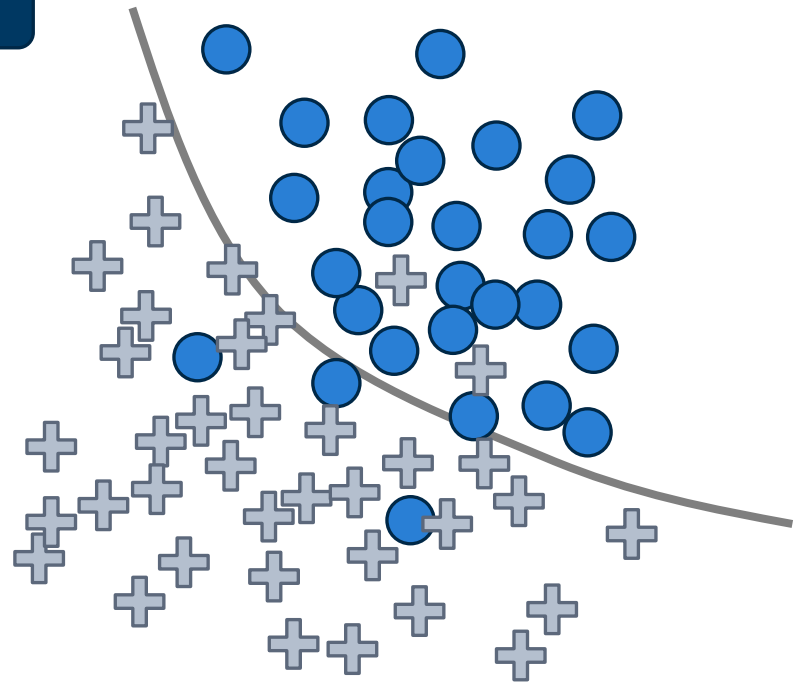


Over-fitting

Does not generalize!

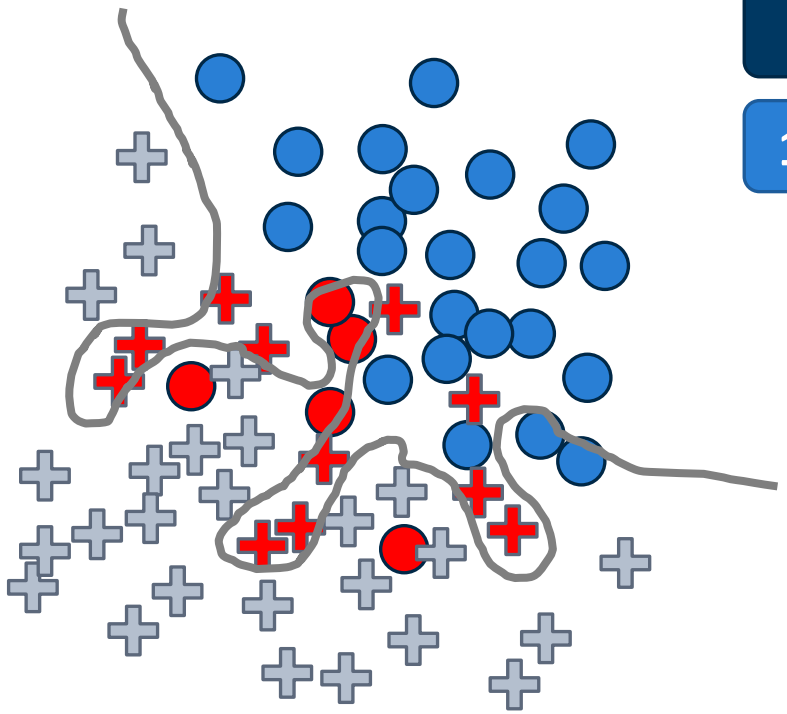


# Errors		# Errors
0	Training set	5
16	Test set	



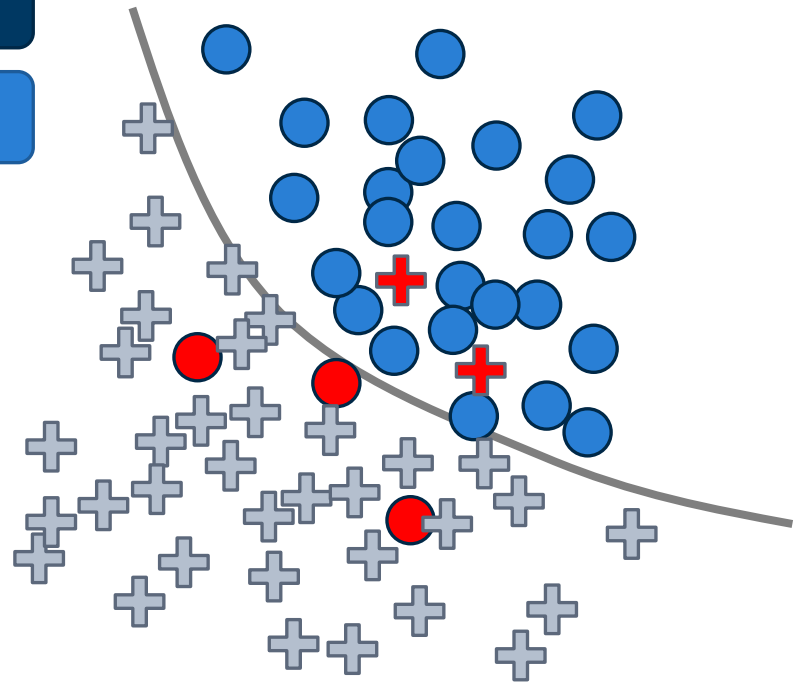
Over-fitting

Does not generalize!



# Errors		# Errors
0	Training set	5
16	Test set	5

Generalizes well



What does this mean in practice?

Goal: Generalization

Machine learning is about generalization, not about optimization

CONTROL MODEL COMPLEXITY

A number of techniques exist:

- Reduce number of parameters
- Regularization
- Early stopping

ESTIMATE GENERALIZATION ERROR

Split dataset into

- **Training set** used to train the algorithm using empirical risk minimization (ERM)
- (**Validation set** for hyperparameter tuning or model selection)
- **Test set** used to estimate the true risk

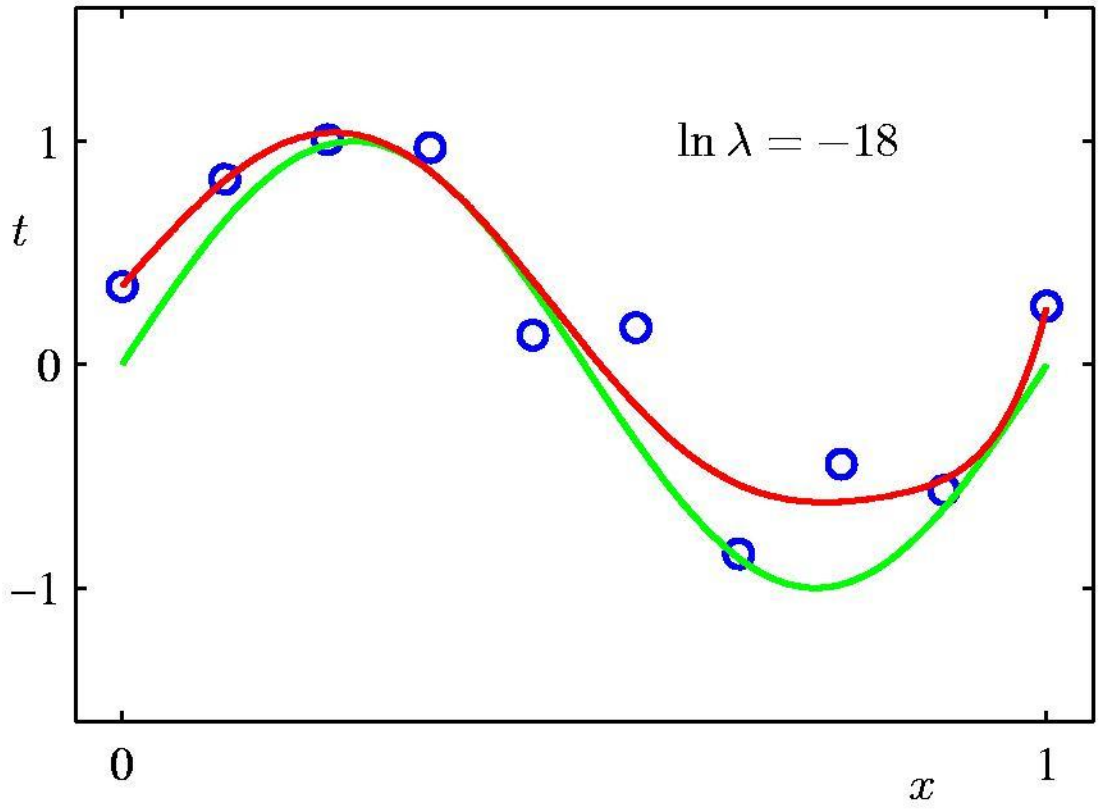
Regularization

Regularization

Penalize large coefficient values

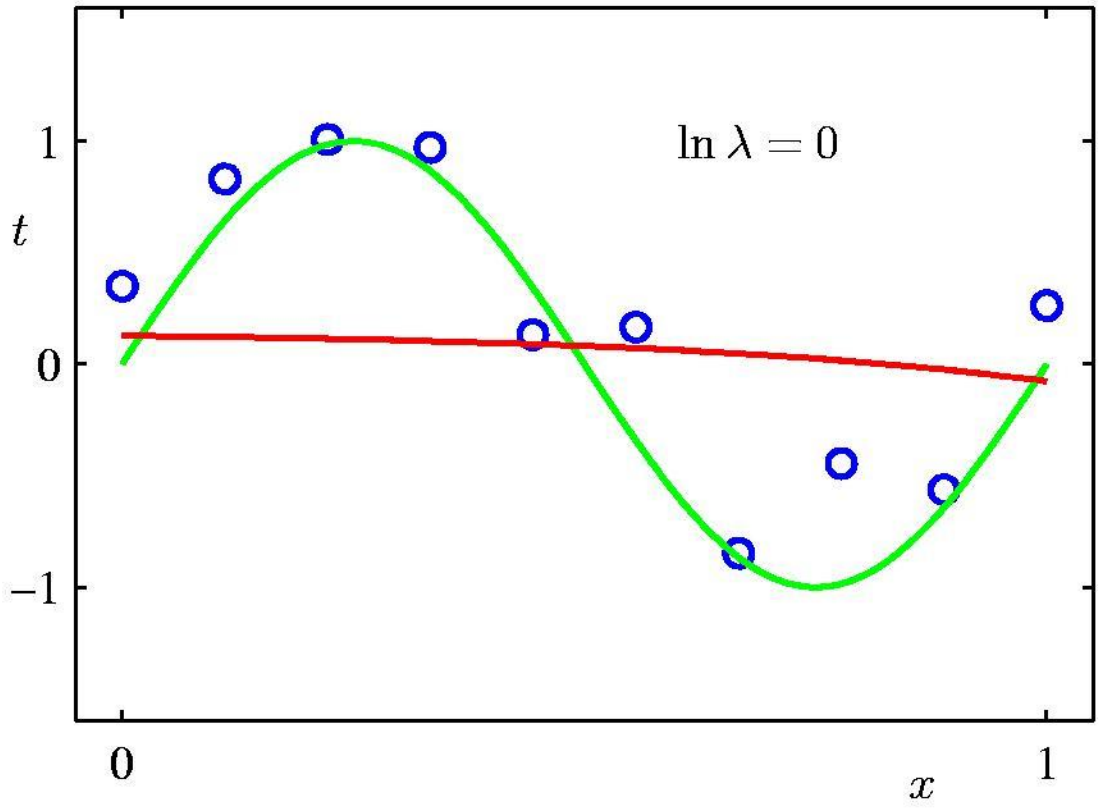
$$\widetilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

Moderate Regularization



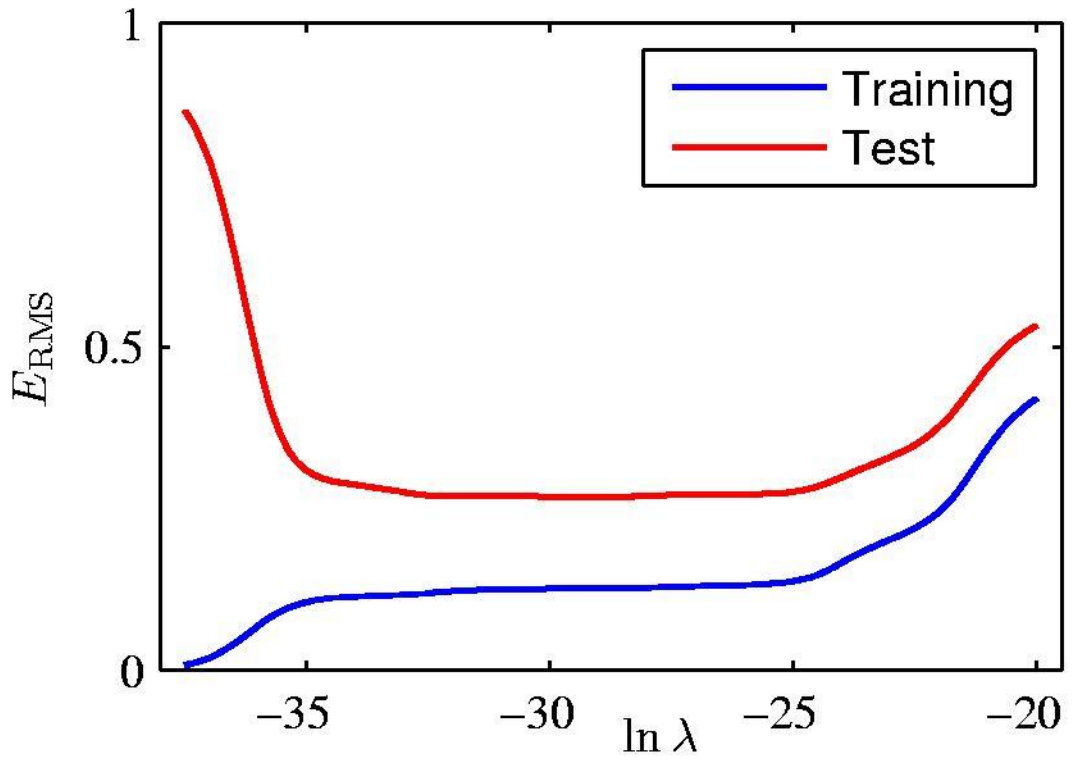
Bishop: Pattern Recognition and Machine Learning

Regularization too strong



Bishop: Pattern Recognition and Machine Learning

Squared error vs. regularization strength



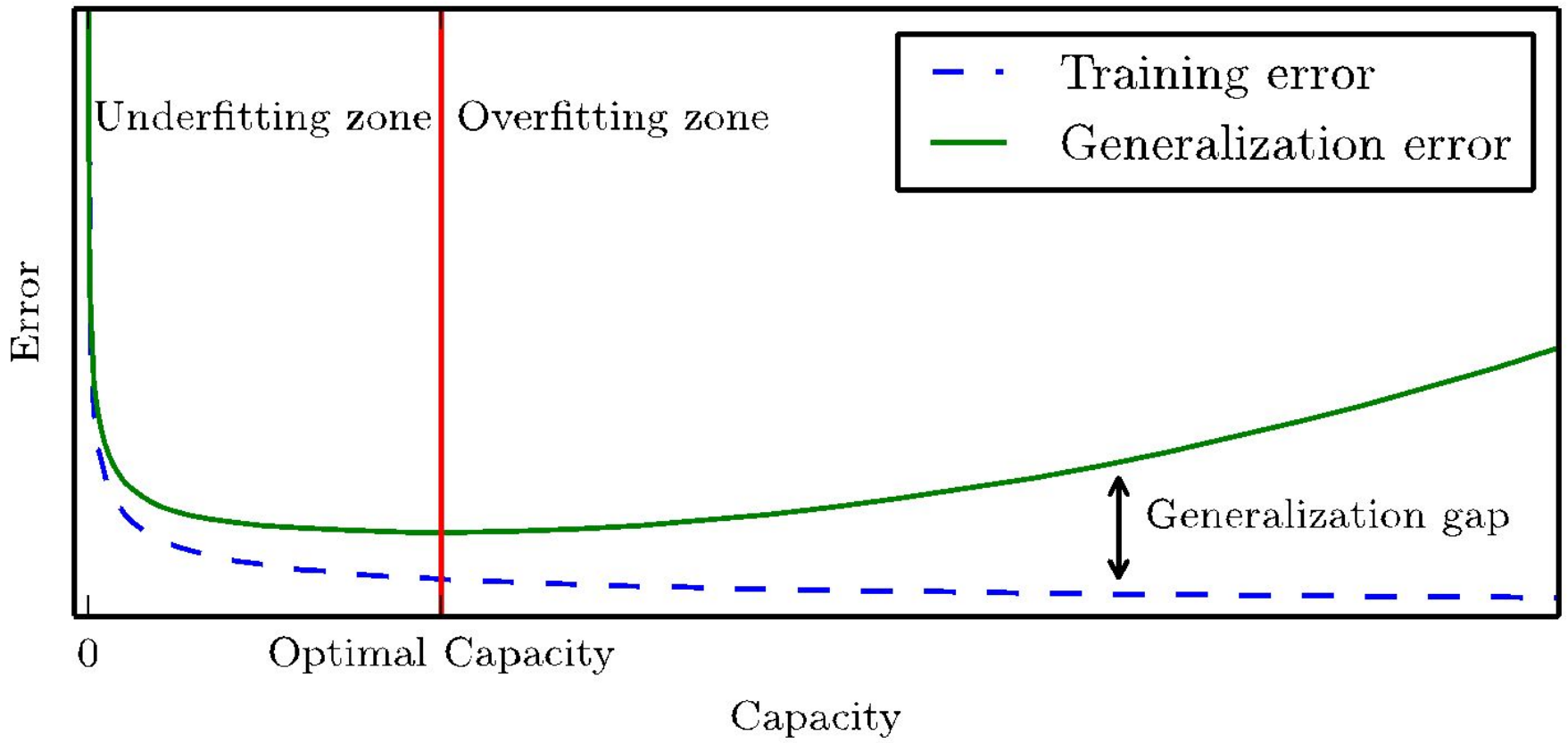
Here we would use a validation set for choosing the best parameter.
The test set has to remain untouched for a final assessment of the risk.

Bishop: Pattern Recognition and Machine Learning

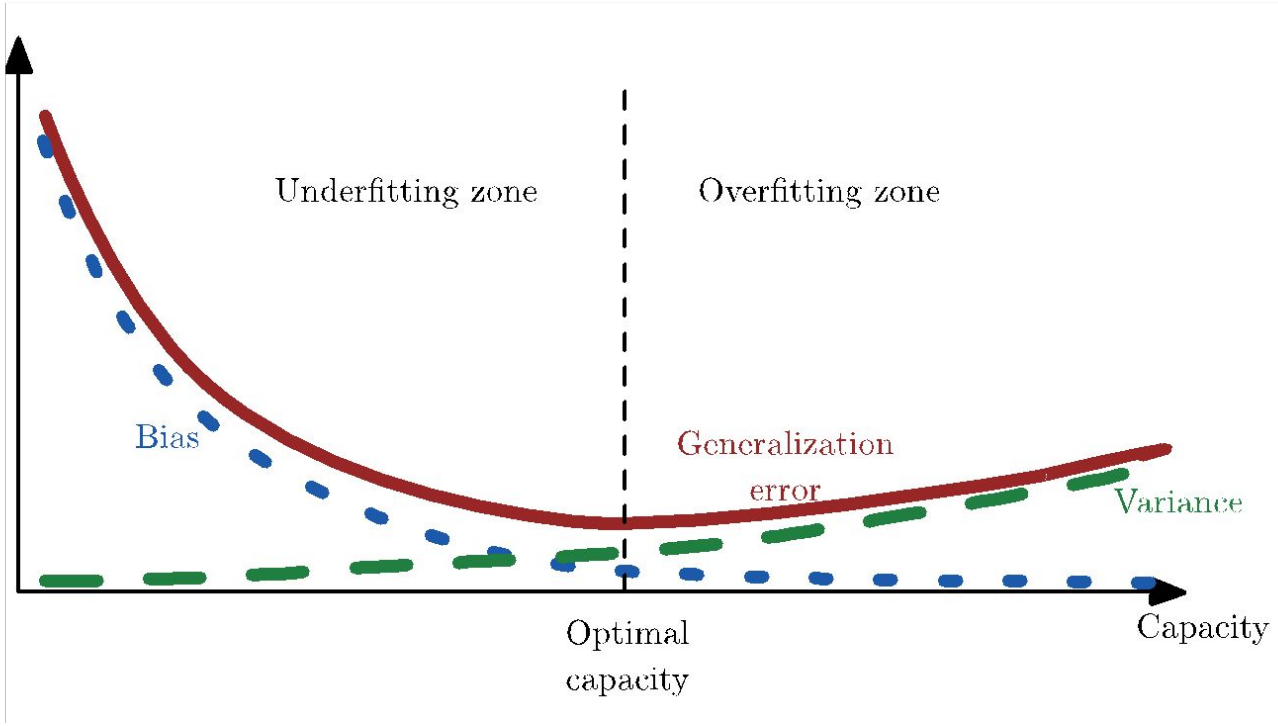
Polynomial Coefficients

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
$w_0^\star$	0.35	0.35	0.13
$w_1^\star$	232.37	4.74	-0.05
$w_2^\star$	-5321.83	-0.77	-0.06
$w_3^\star$	48568.31	-31.97	-0.05
$w_4^\star$	-231639.30	-3.89	-0.03
$w_5^\star$	640042.26	55.28	-0.02
$w_6^\star$	-1061800.52	41.32	-0.01
$w_7^\star$	1042400.18	-45.95	-0.00
$w_8^\star$	-557682.99	-91.53	0.00
$w_9^\star$	125201.43	72.68	0.01

Generalization and capacity



Bias and variance

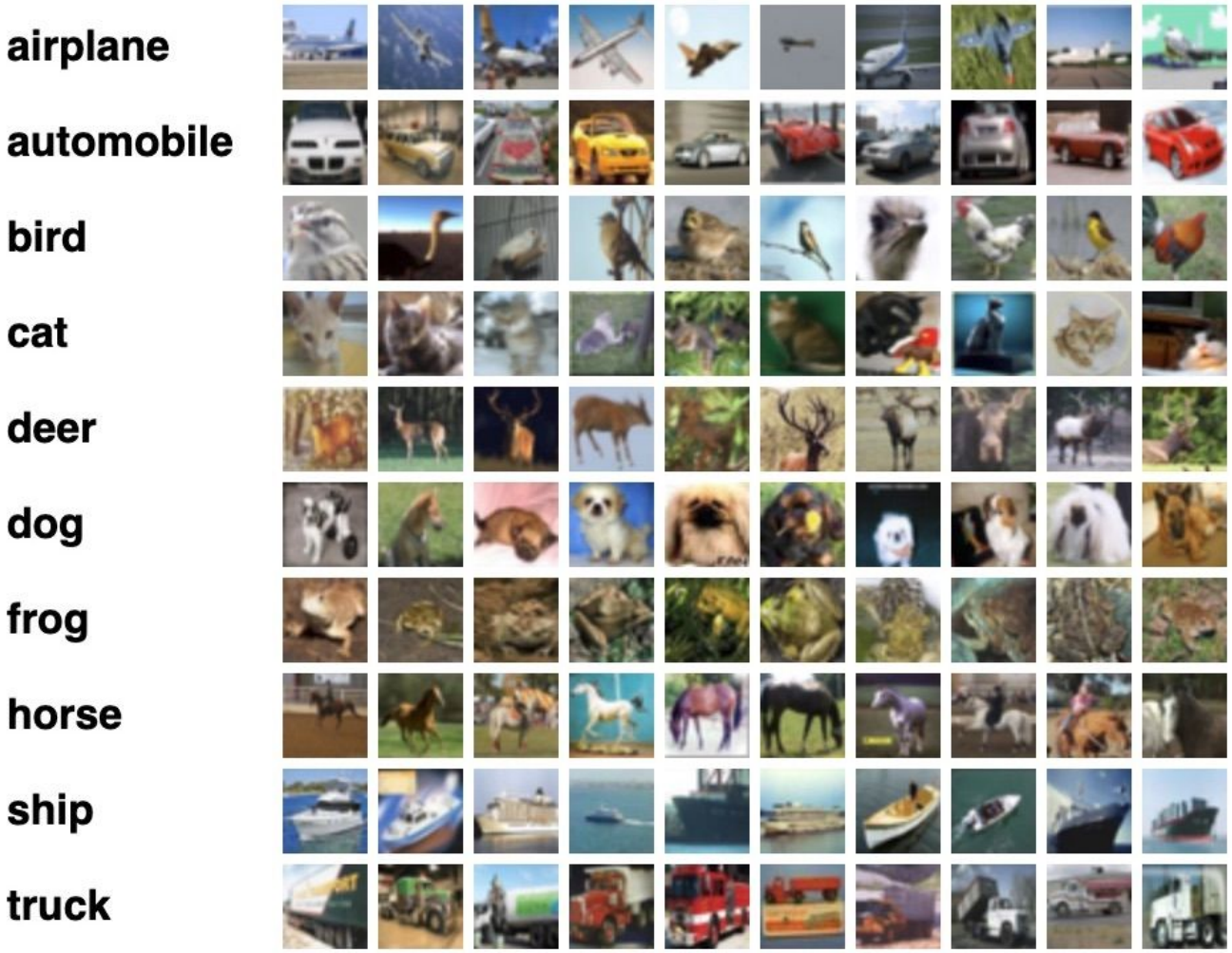


- Bias: systematic errors
  - e.g. too simple model for data complexity
- Variance: error from sensitivity to small fluctuations in the training set.

Datasets, Splits & Validation

Datasets in practice

CIFAR10 Dataset

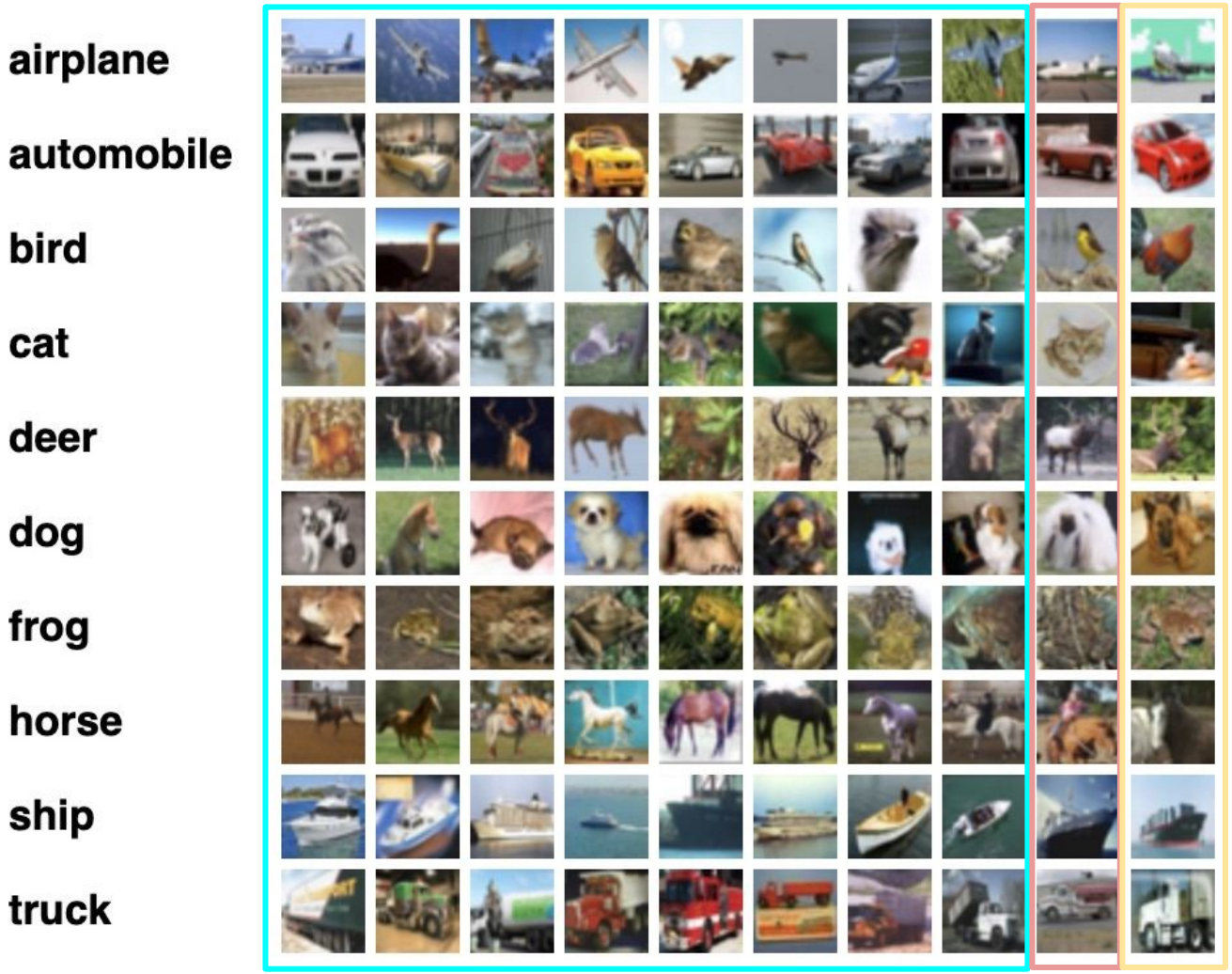


Dataset for supervised learning:

- X: input / features
  - Vision:
    - raw pixels of the image (Deep Learning)
    - features derived from the image data, e.g. via filters
- Y: output / targets
  - The class labels: airplane, automobile, …

Datasets in practice

CIFAR10 Dataset

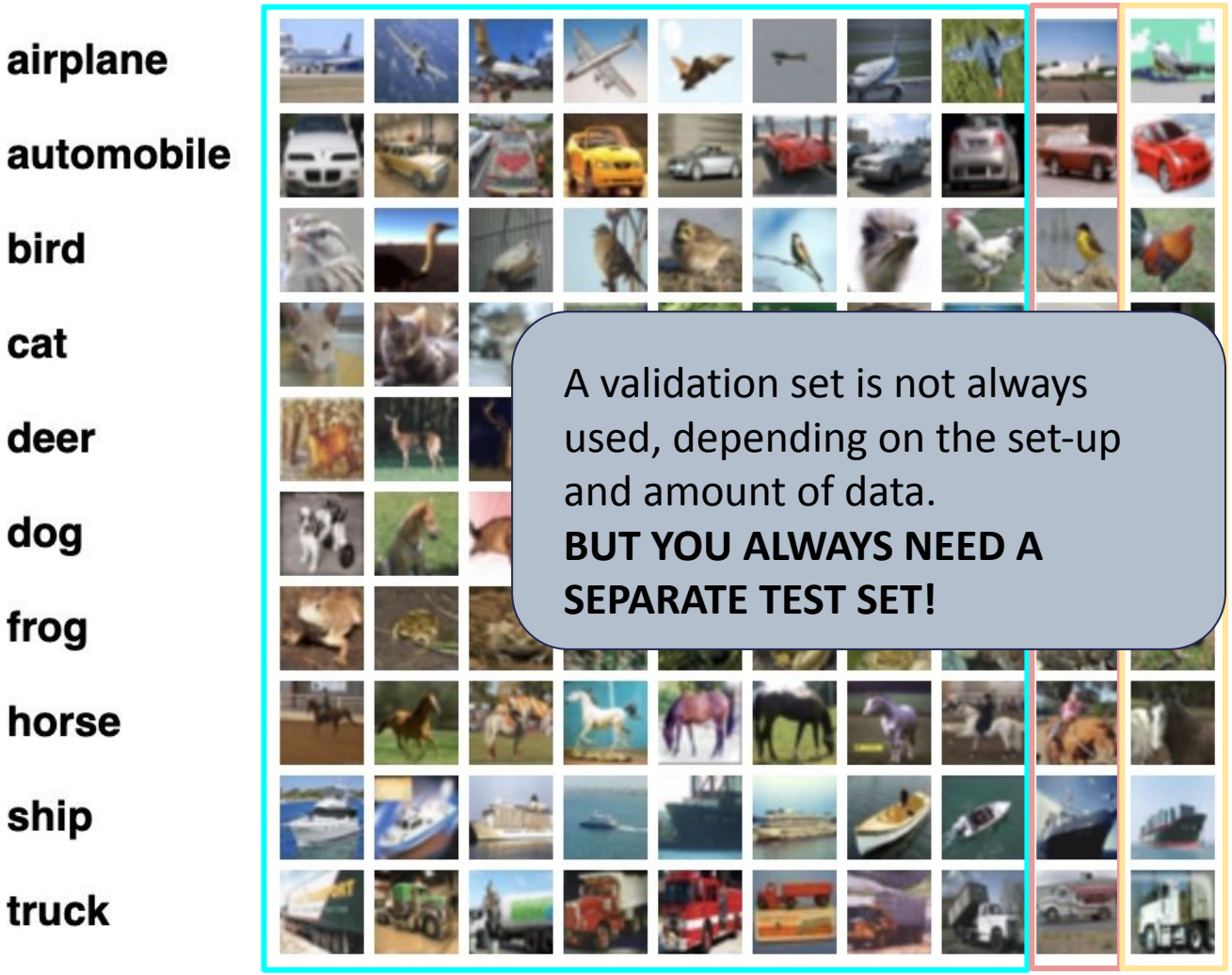


Datasplits for supervised learning

- Training set:
  - For training the ML / DL algorithm.
  - ca. 60-80% of the data.
- Validation set
  - For finding good hyperparameters (e.g. regularization strength or learning rate)
  - ca. 10-20% of the data.
- Test set:
  - ca. 10-20% of the data.
  - for evaluating the performance on unseen data

Datasets in practice

CIFAR10 Dataset

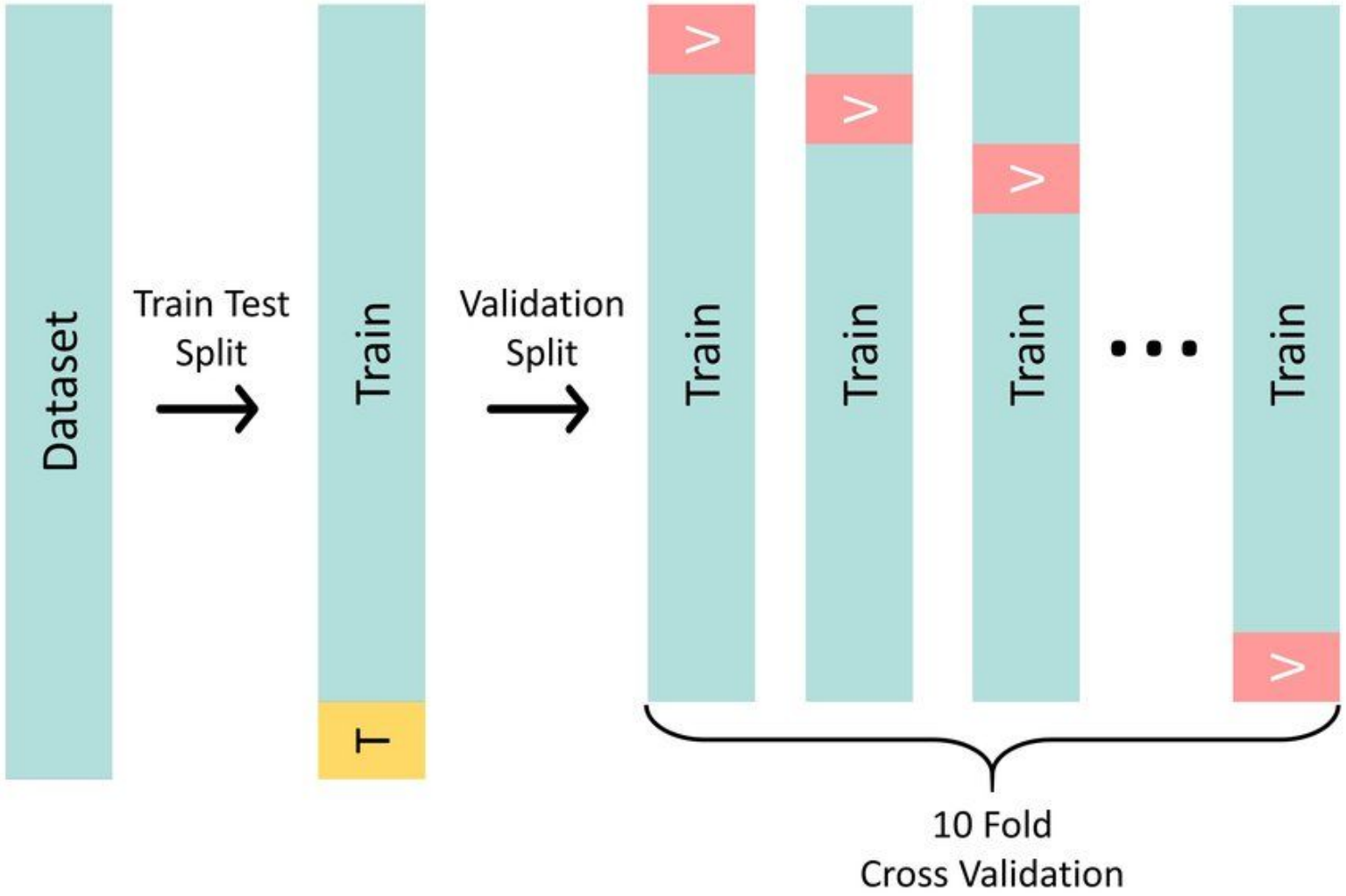


A validation set is not always used, depending on the set-up and amount of data.
BUT YOU ALWAYS NEED A SEPARATE TEST SET!

Datasplits for supervised learning

- Training set:
  - For training the ML / DL algorithm.
  - ca. 60-80% of the data.
- Validation set
  - For finding good hyperparameters (e.g. regularization strength or learning rate)
  - ca. 10-20% of the data.
- Test set:
  - ca. 10-20% of the data.
  - for evaluating the performance on unseen data

Cross-validation

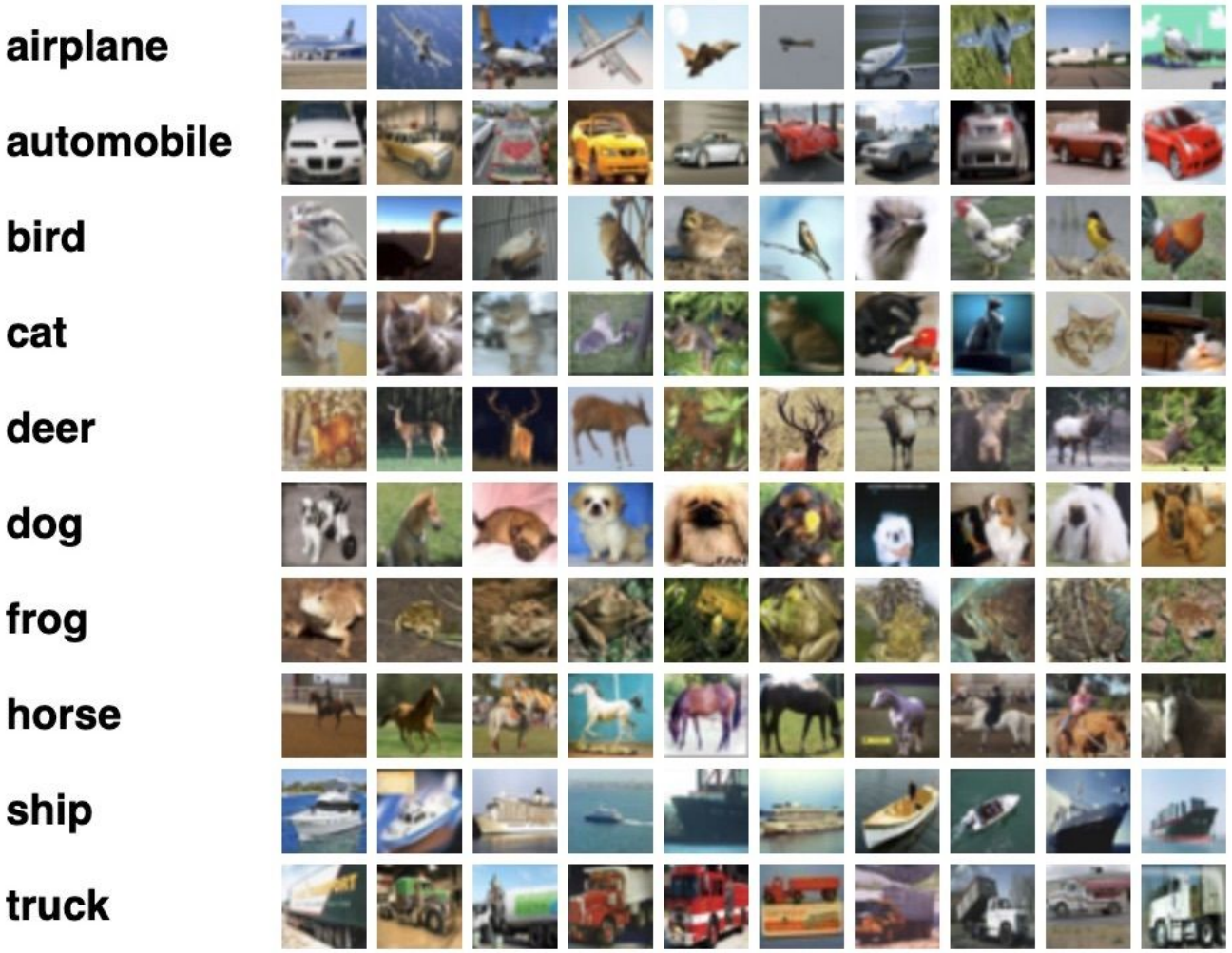


- Eliminates potential bias due to fixed train and validation set
- Use for reliable hyperparameter finding
- Needs more trainings -> we will not do this in the exercises due to long training times

Deep Learning

DL Basics

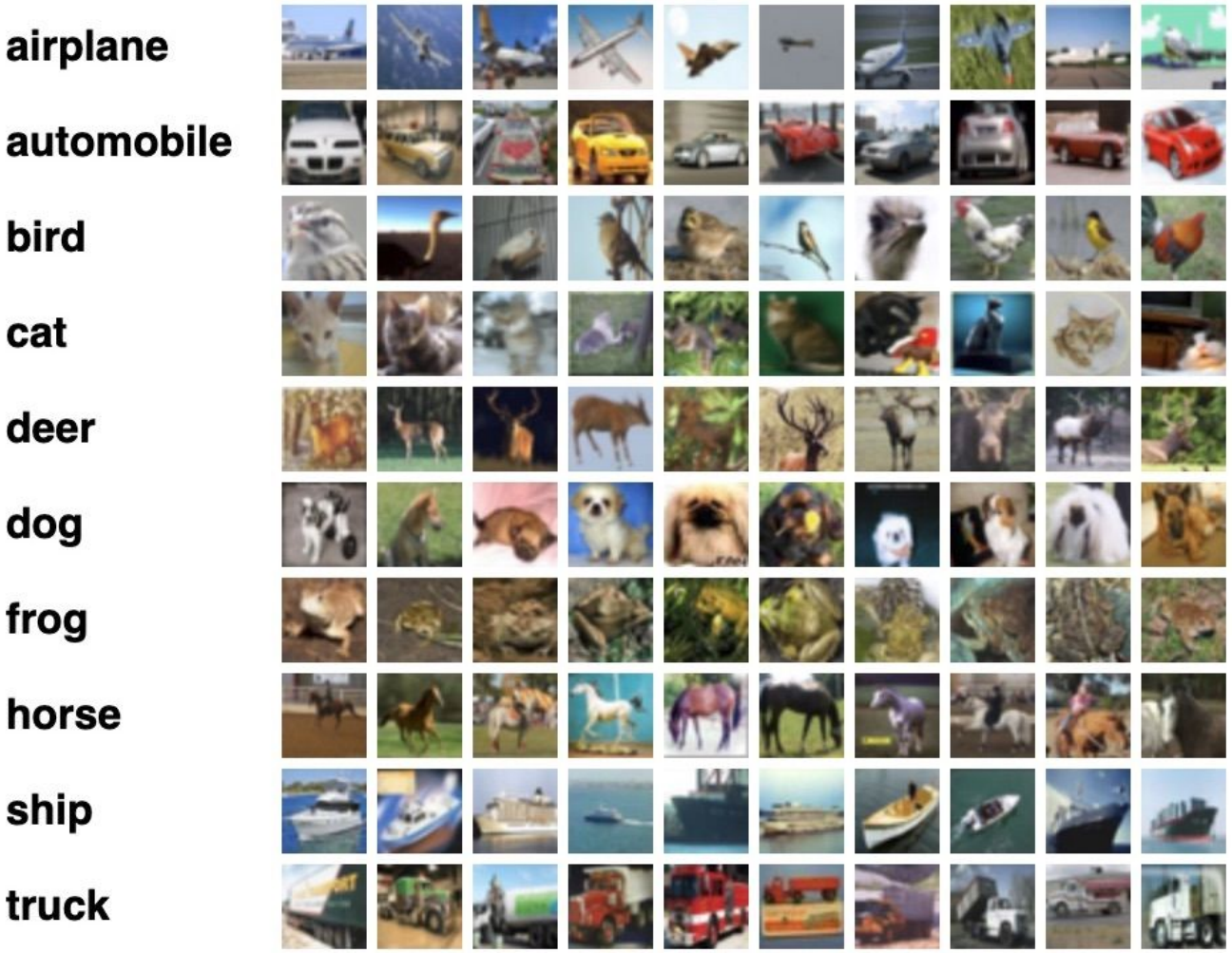
Why deep learning?



Problem when applying e.g. a Random Forest to this data:

- Does not work well on pixel level: mapping from data to classes is too complex for that
- How can we define good features for the classification task?
- Can use edge detectors and other filters.
- But: feature designing is difficult and sub-optimal features limit the performance.

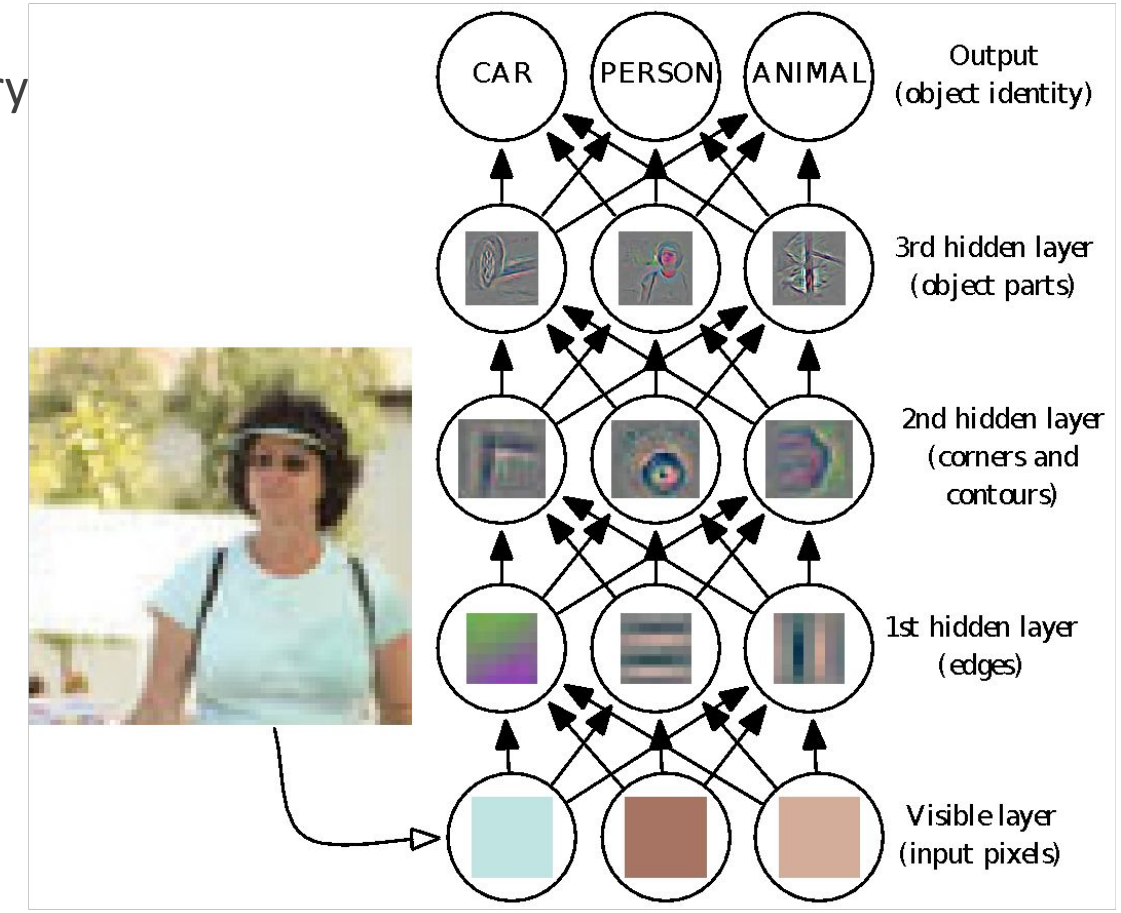
Why deep learning?



Deep Learning:
Learn features and decision from the data!

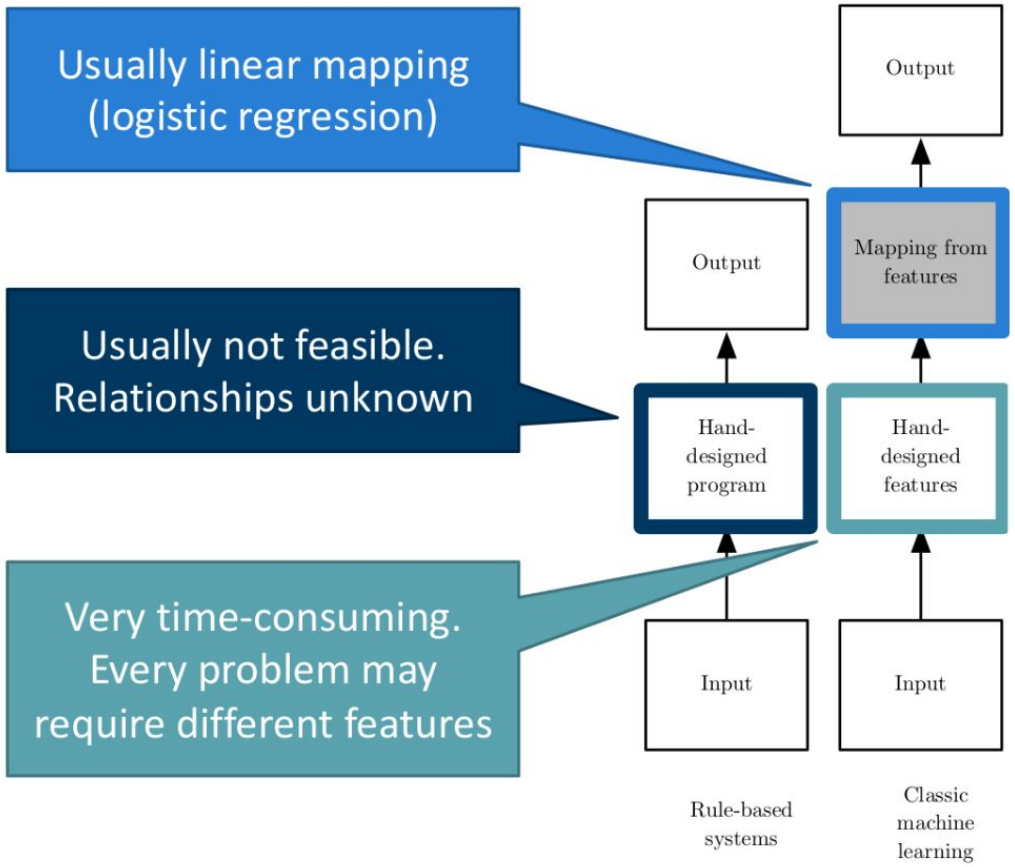
Why deep?

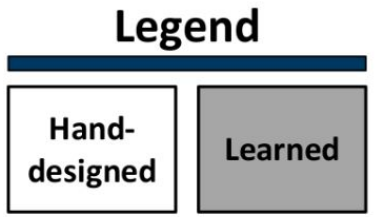
Relationship between raw data (here: pixels) and class label is very complex



Quelle: Goodfellow, Bengio, Courville 2016

Learning everything





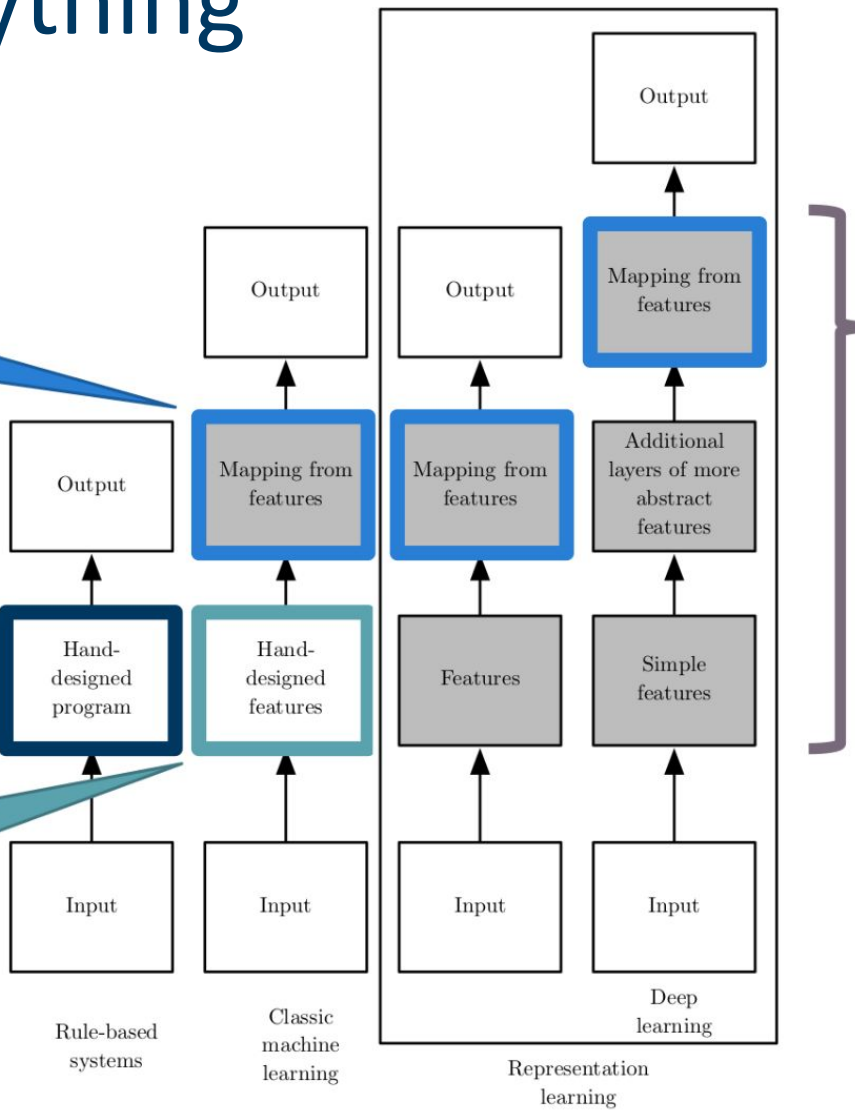
Quelle: Goodfellow, Bengio, Courville 2016

Learning everything

Usually linear mapping (logistic regression)

Usually not feasible. Relationships unknown

Very time-consuming. Every problem may require different features



"End-to-end learning"

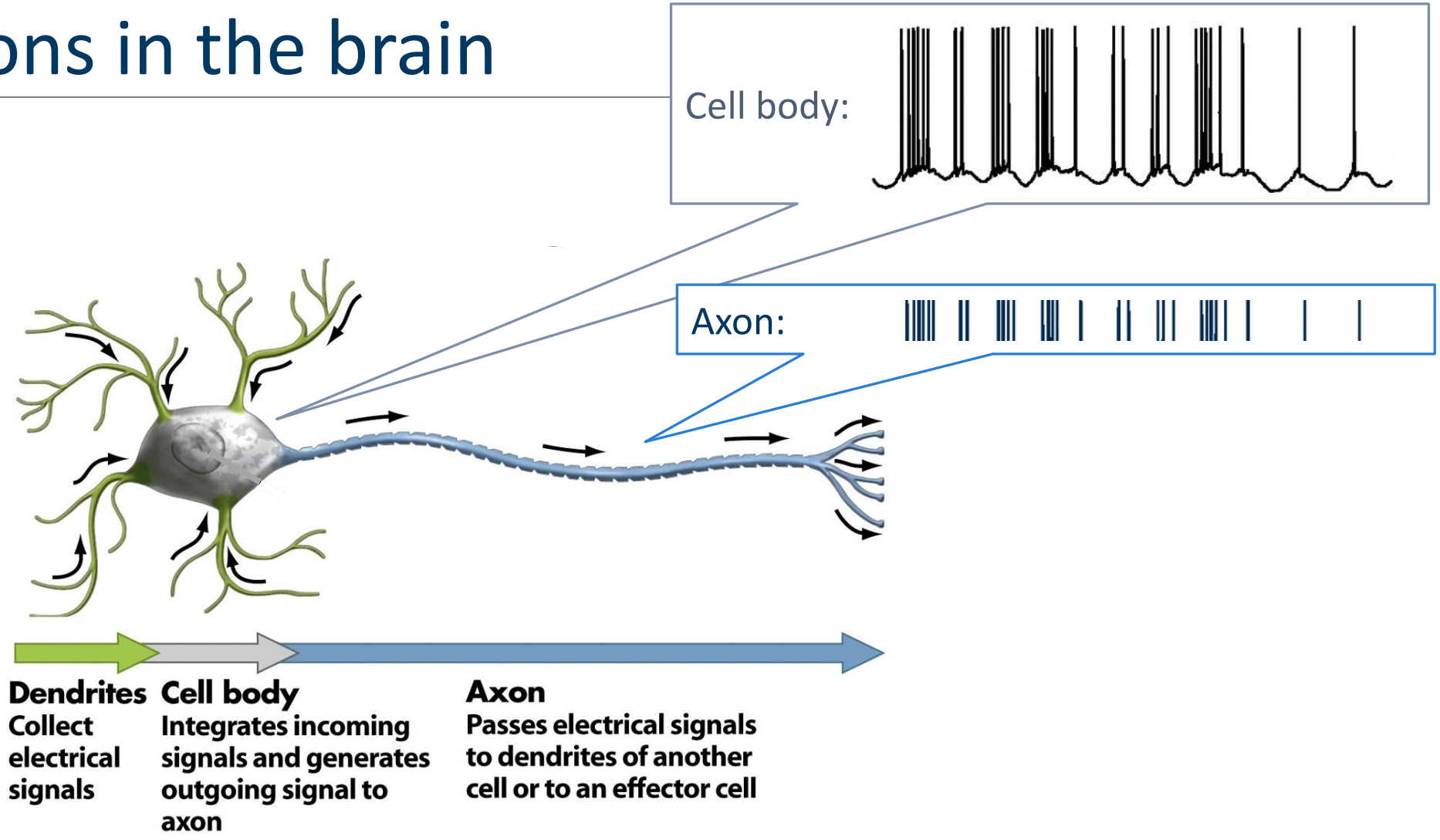
Legend

Hand-designed | Learned

Quelle: Goodfellow, Bengio, Courville 2016

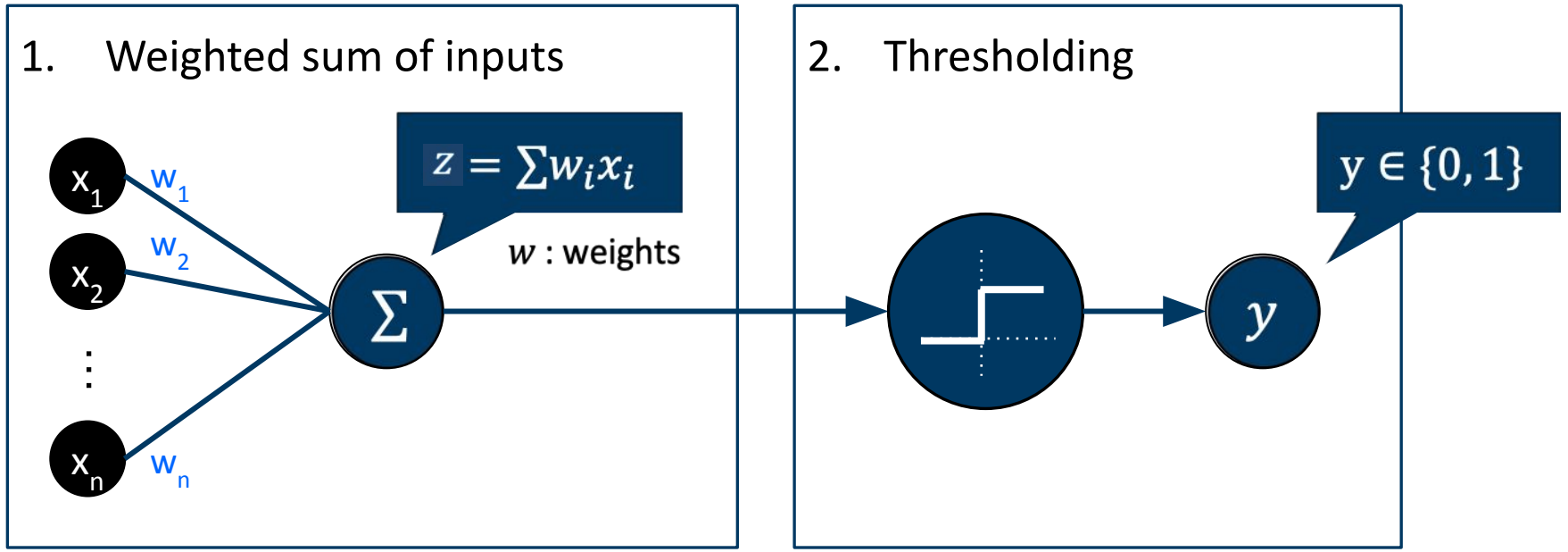
The Perceptron

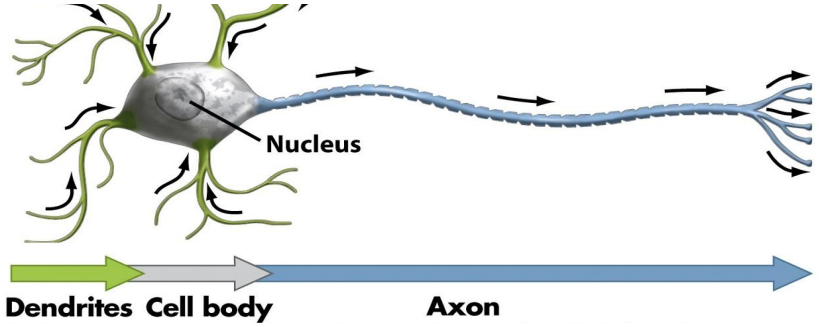
Neurons in the brain



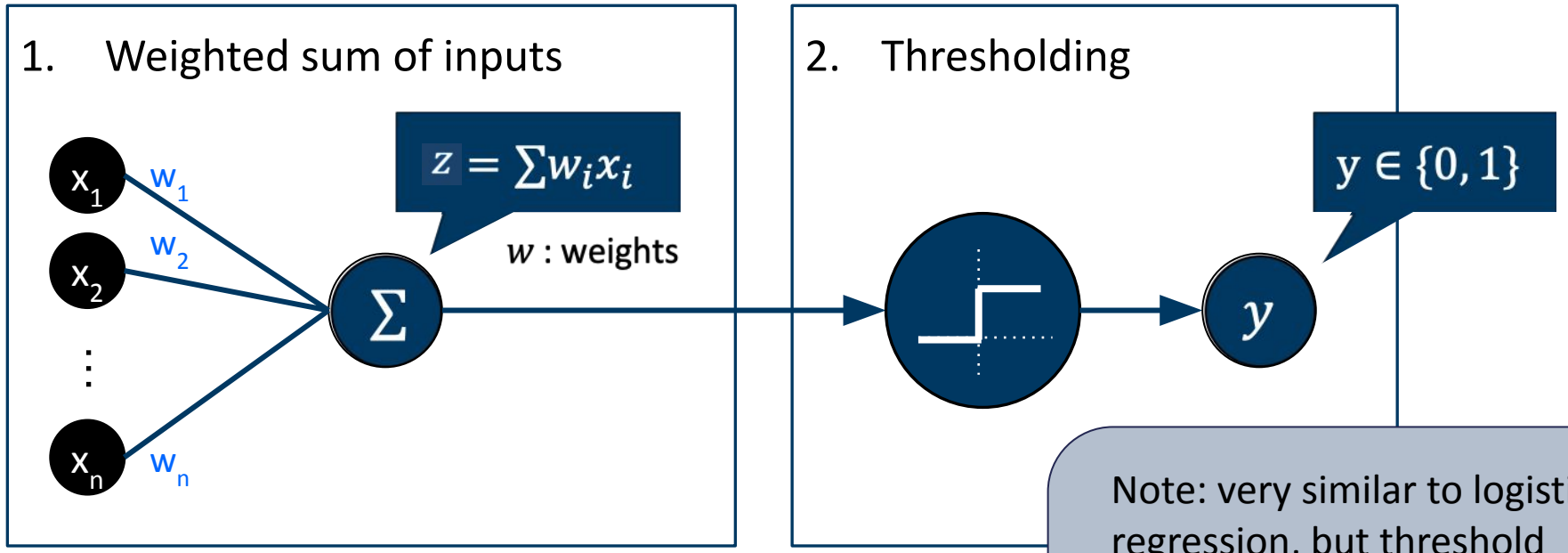
https://www.thinglink.com/scene/762090261831483394

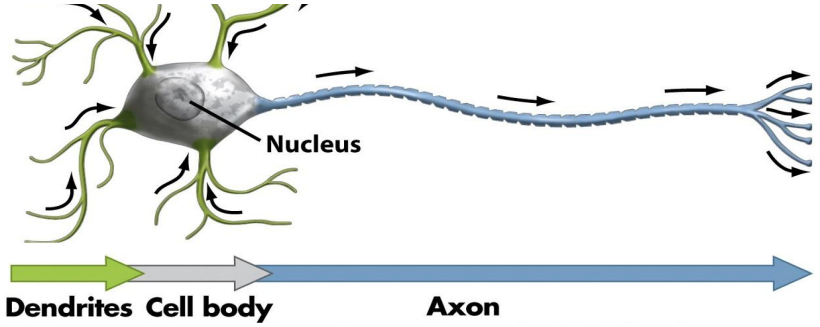
The Perceptron (Rosenblatt 1958)





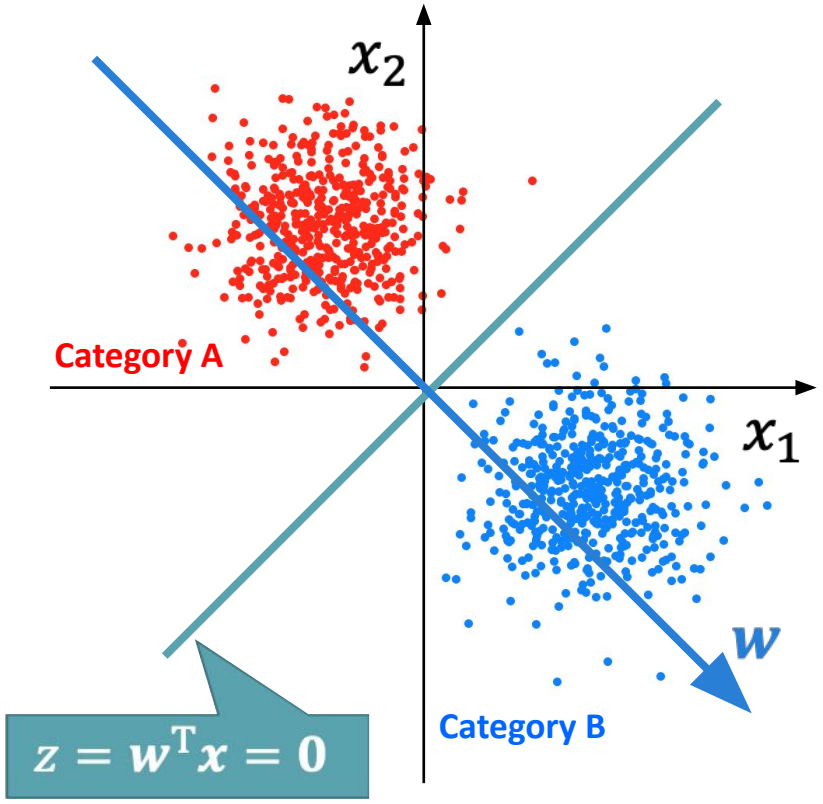
The Perceptron (Rosenblatt 1958)

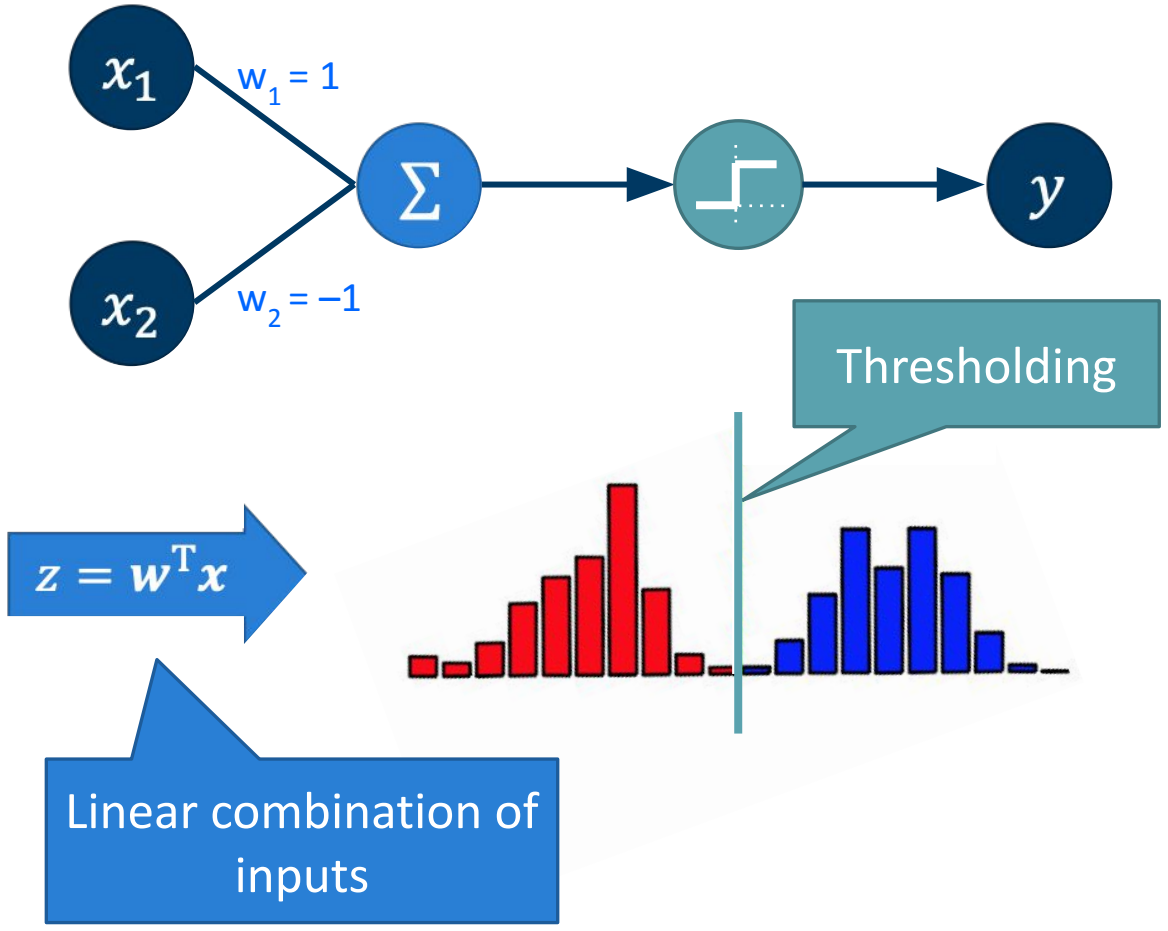




Note: very similar to logistic regression, but threshold instead of sigmoid.
Perceptron: classification
Logistic reg.: prob. classification

Perceptron solves linearly separable problems





Perceptron learning algorithm

Algorithm

$\boldsymbol{w} \leftarrow \mathbf{0}$

Iterate over training examples $\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)$ until convergence:

threshold!

$\hat{y}^{(i)} \leftarrow \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x}^{(i)}$

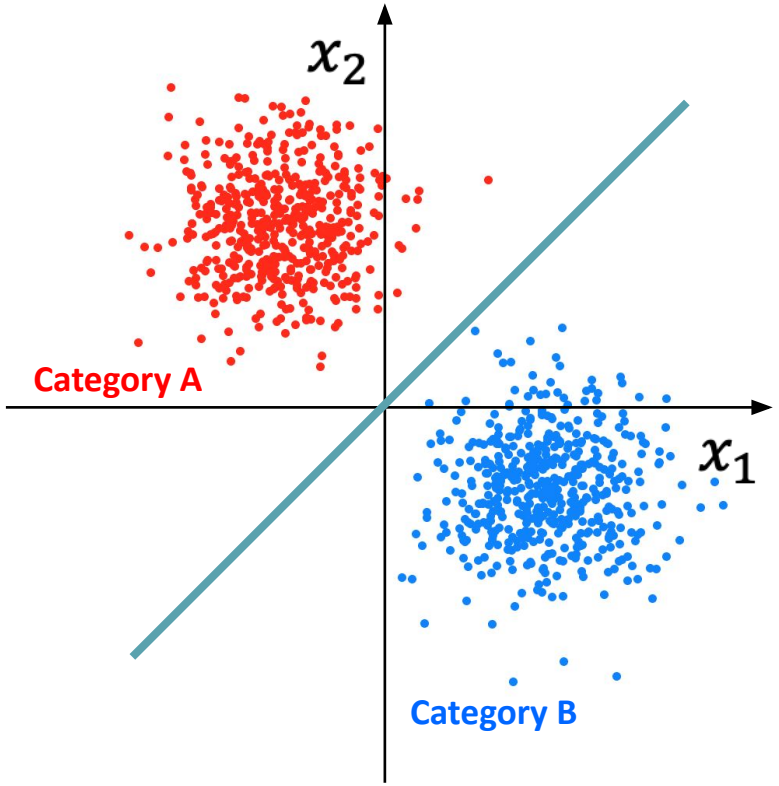
$e \leftarrow y^{(i)} - \hat{y}^{(i)}$

$\boldsymbol{w} \leftarrow \boldsymbol{w} + e \cdot \boldsymbol{x}$

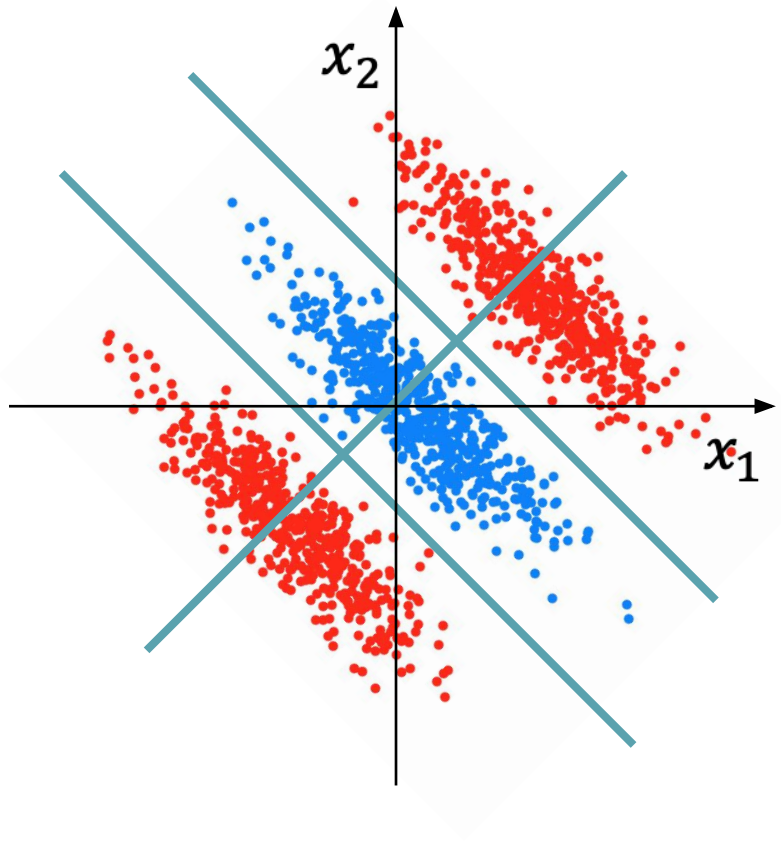
If all training examples can be classified correctly (problem is linearly separable), this algorithm provably converges to a correct solution in a finite number of steps.

Linear (in)separability

Linearly separable

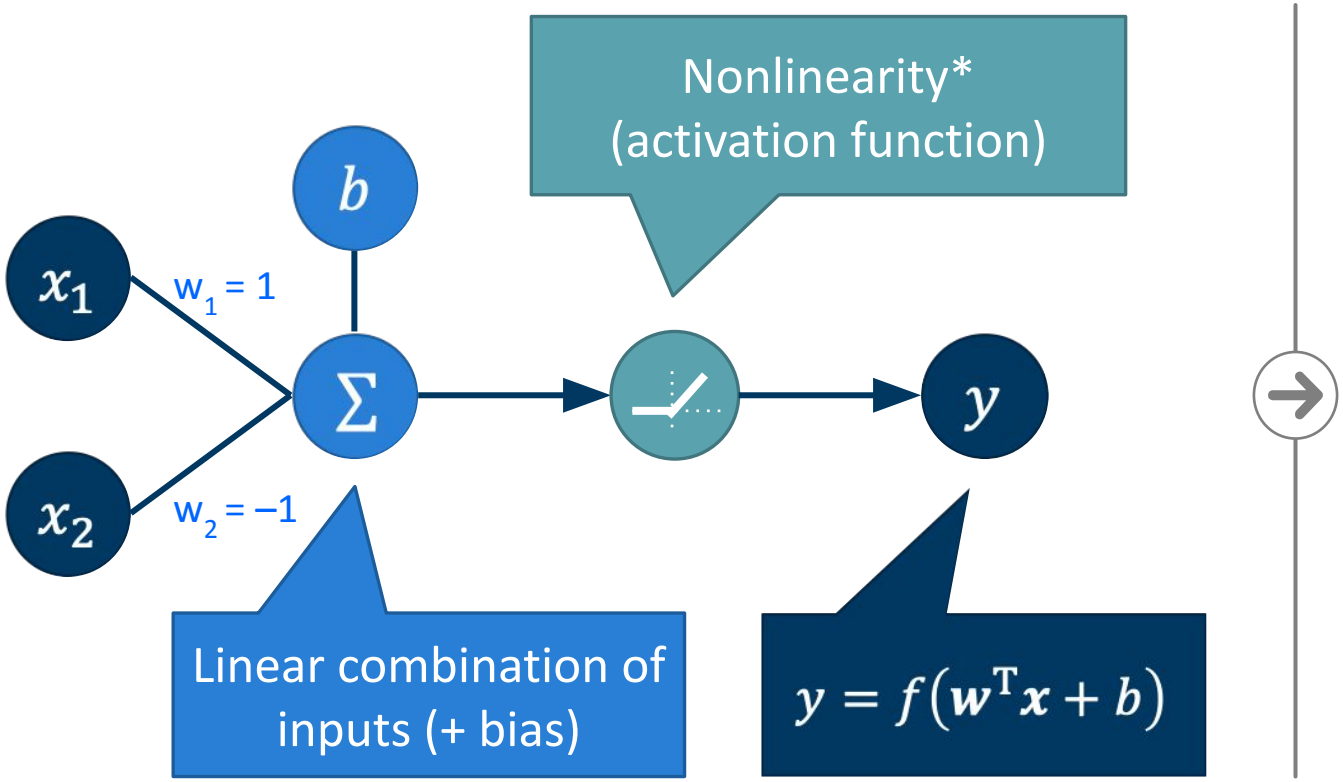


Not linearly separable

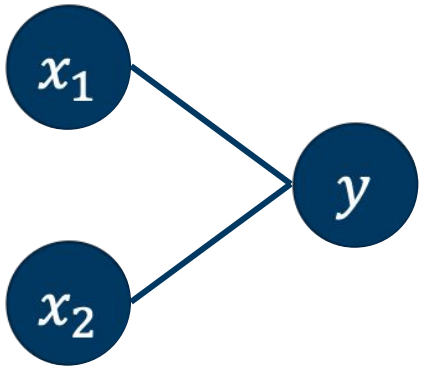


Multi-layer perceptron

Perceptron: more generally



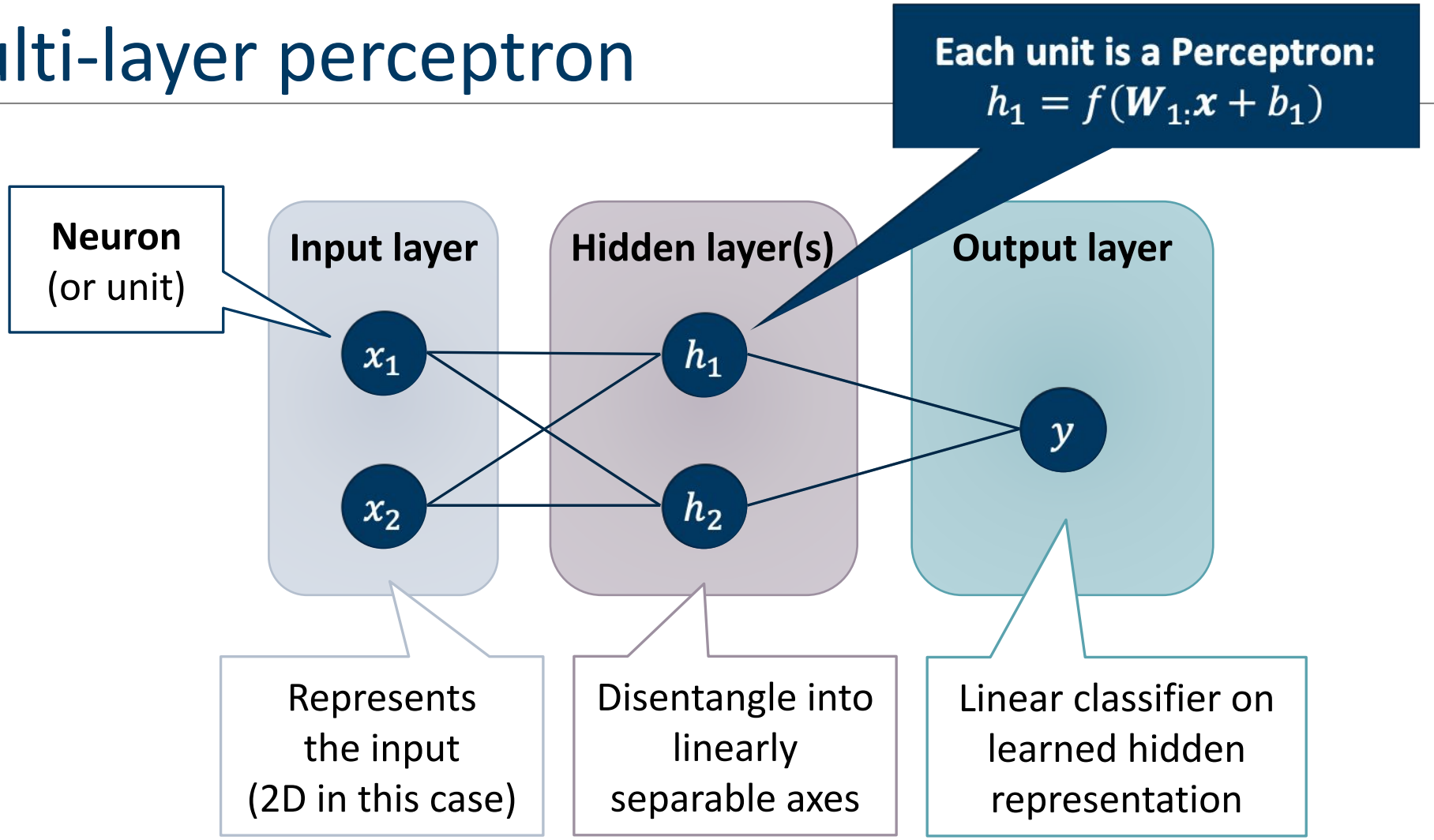
Simplified representation



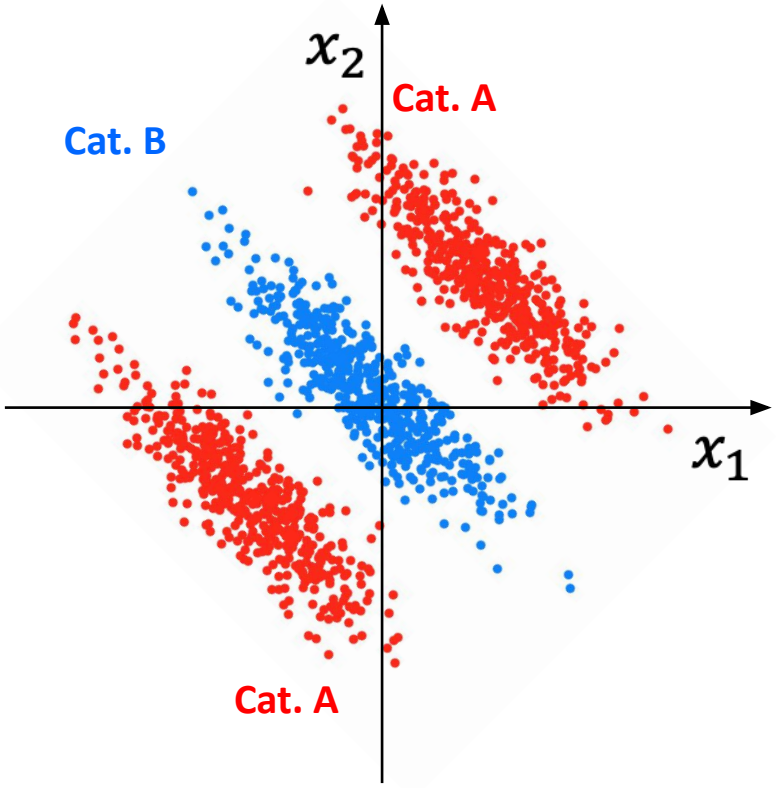
Linear combination and nonlinear activation function are implicit in this diagram

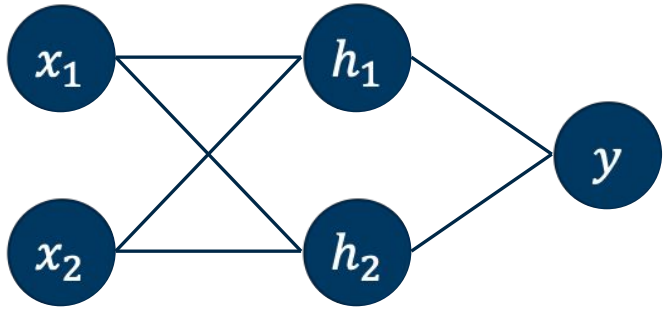
* Original perceptron used thresholding as activation function.
Now: "activation function" is used more broadly for different nonlinearities.

Multi-layer perceptron

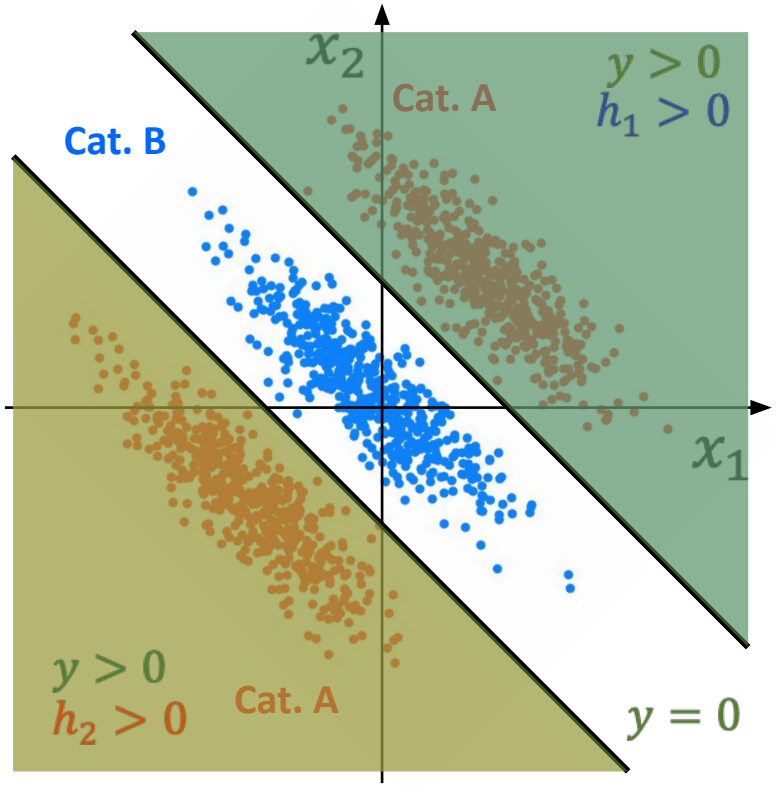


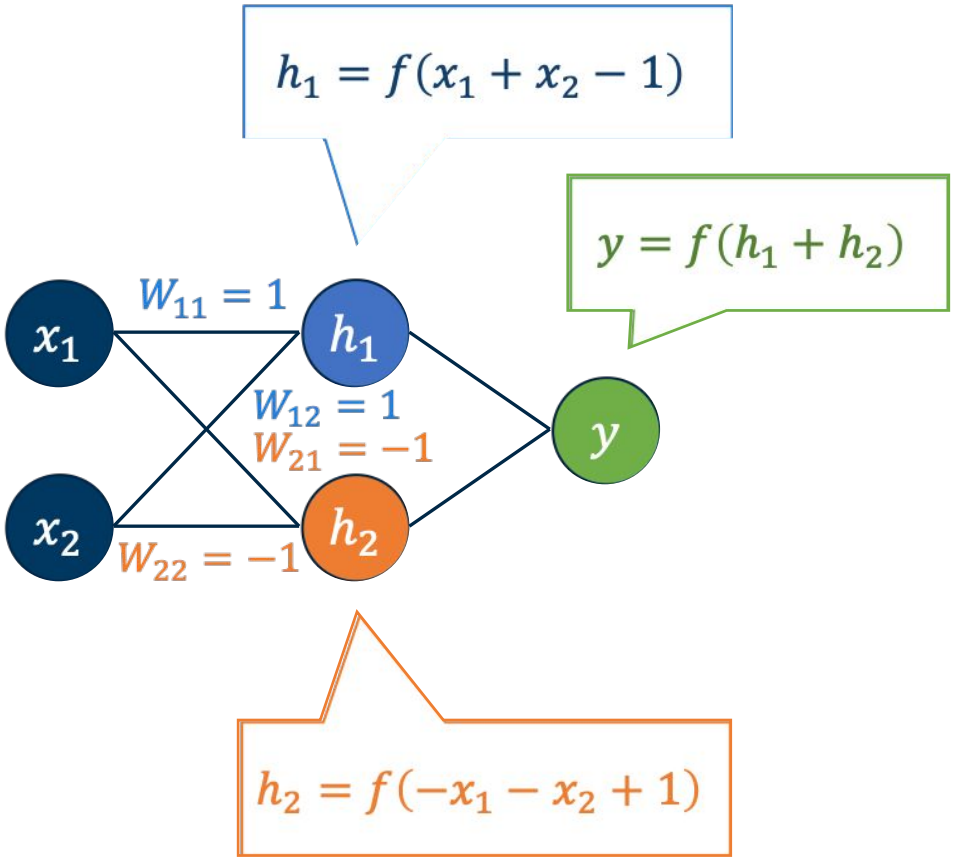
Solving toy problem with a two-layer perceptron





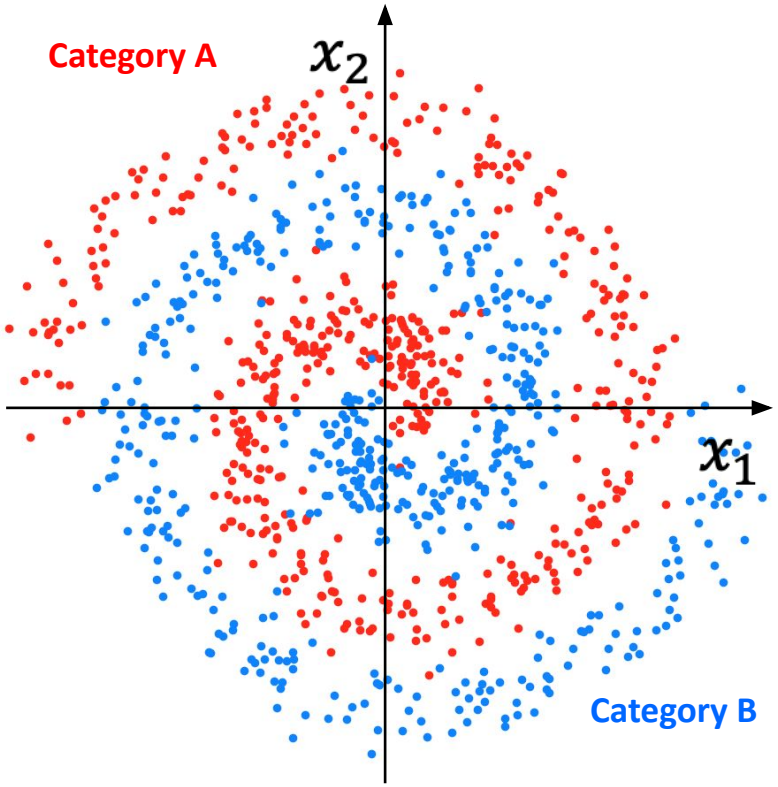
Solving toy problem with a two-layer perceptron





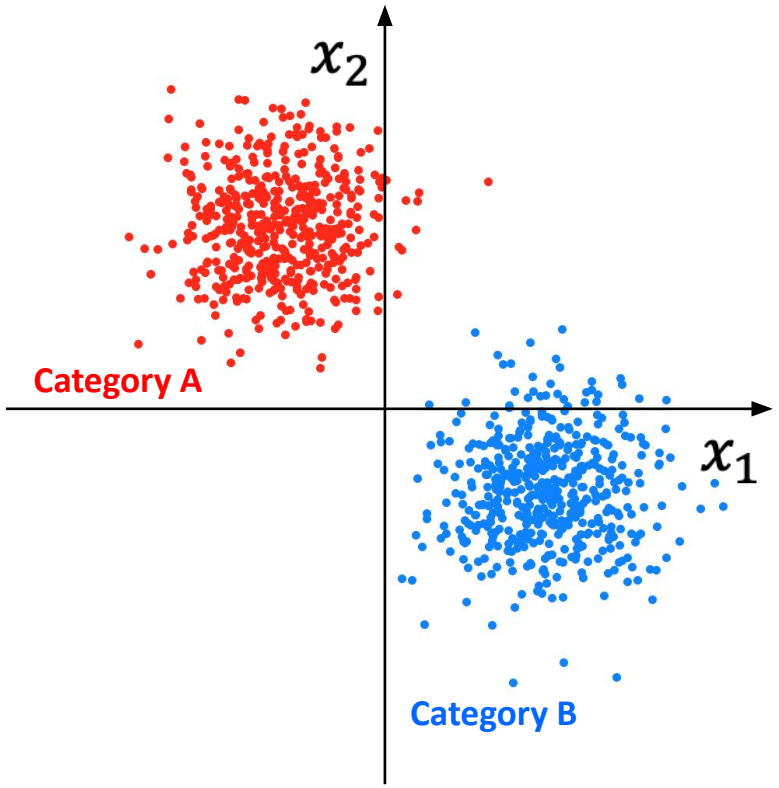
Untangling the manifold: More complex case

Entangled -> Not linearly separable



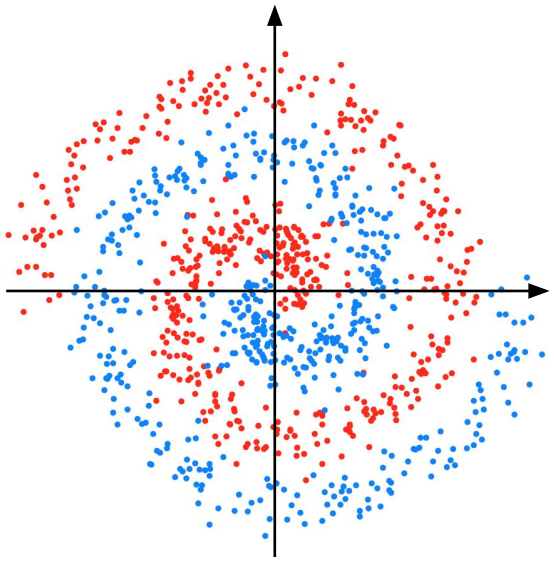
Disentangle through multiple hidden layers

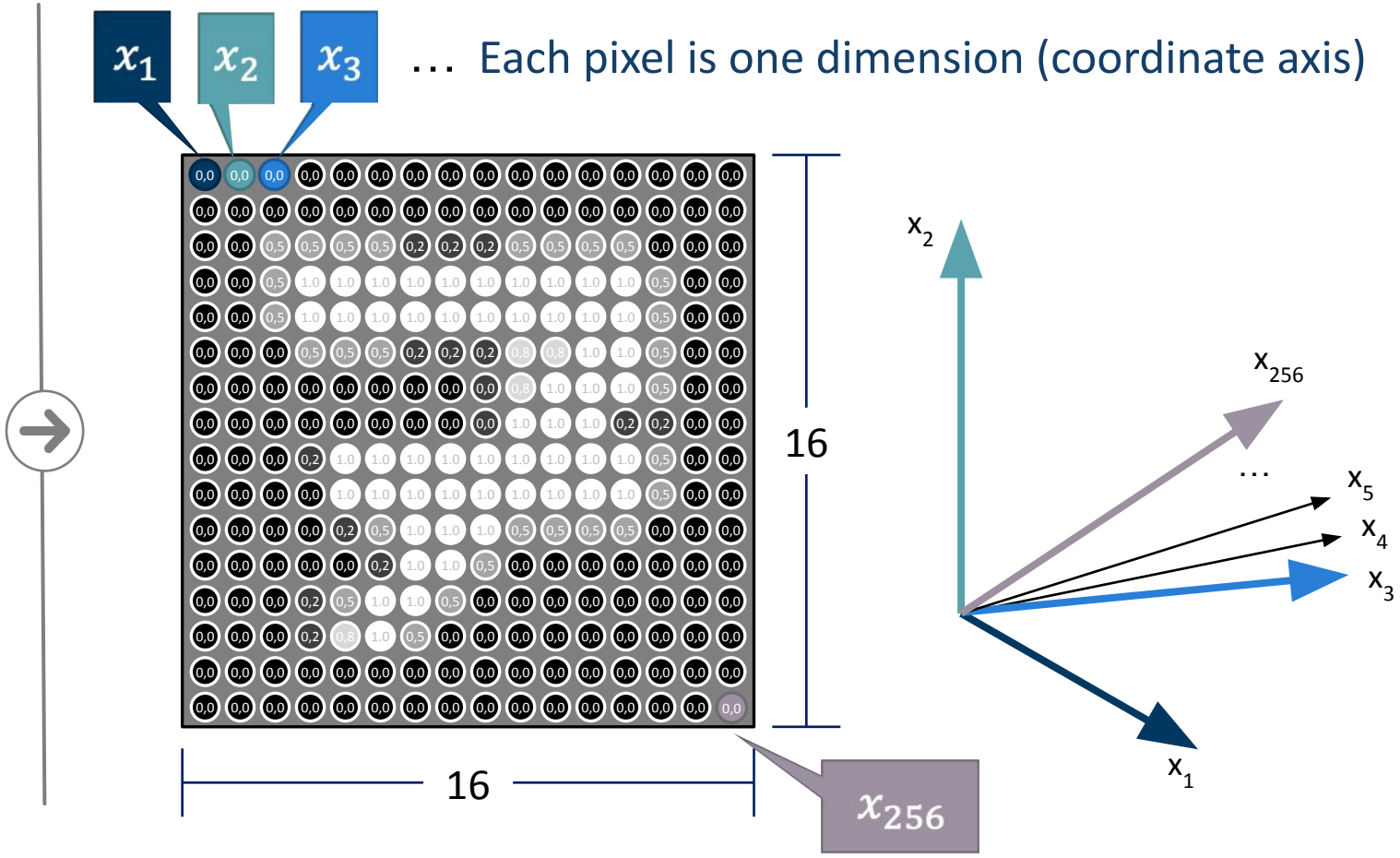
Disentangled -> Linearly separable



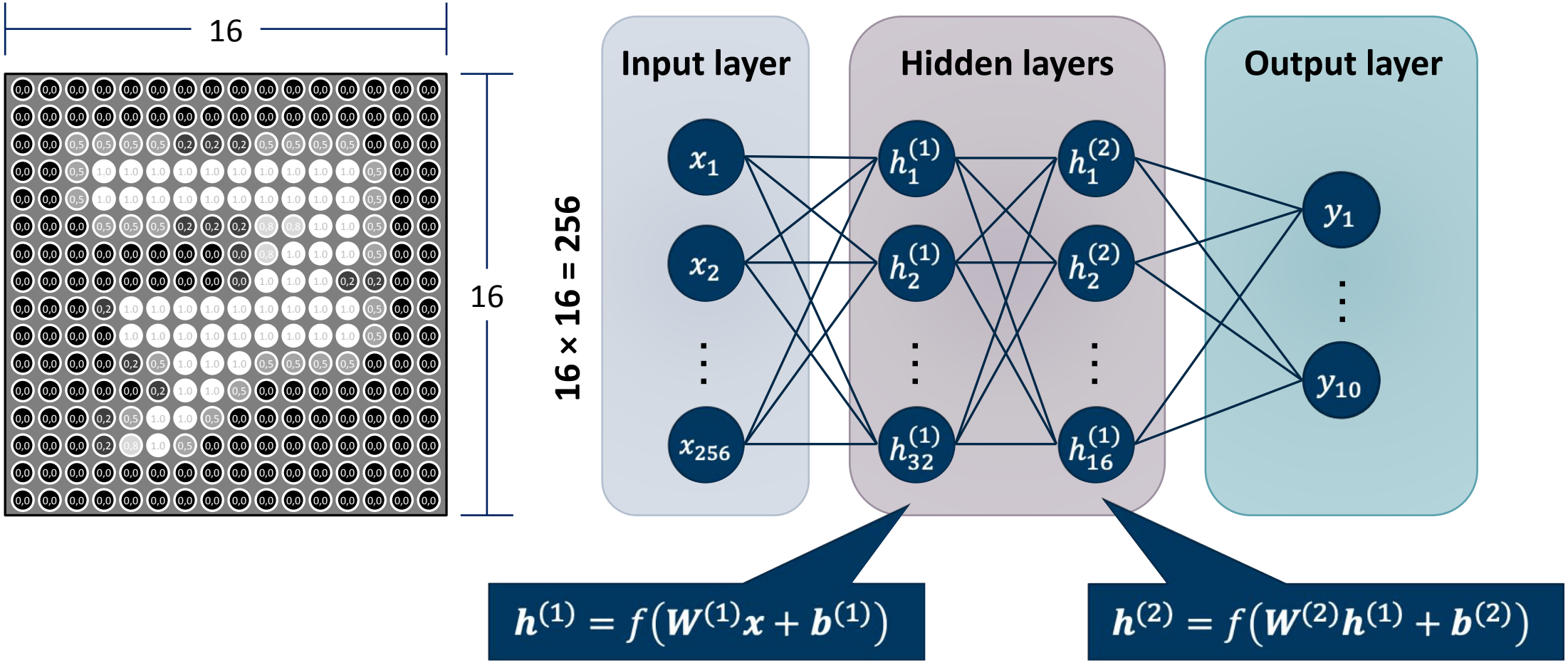
2D to many-D: same principle, hard to visualize

2D toy example so far

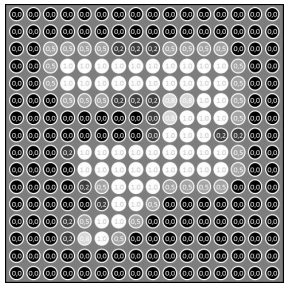


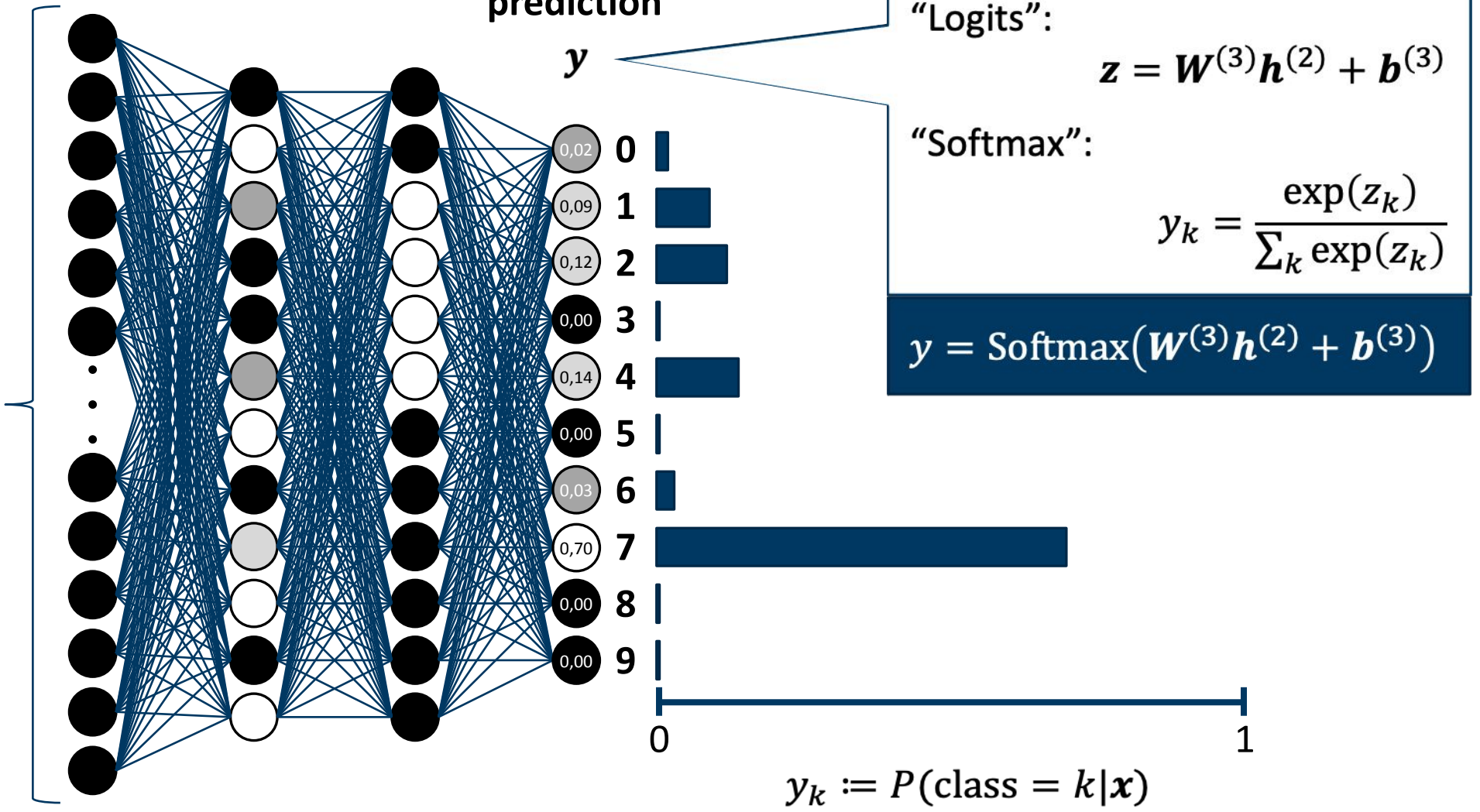


Multi-layer perceptron (MLP) for classifying handwritten digits

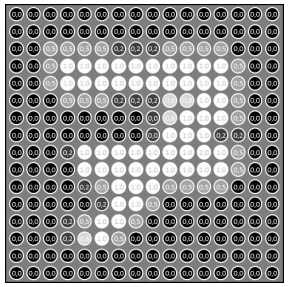


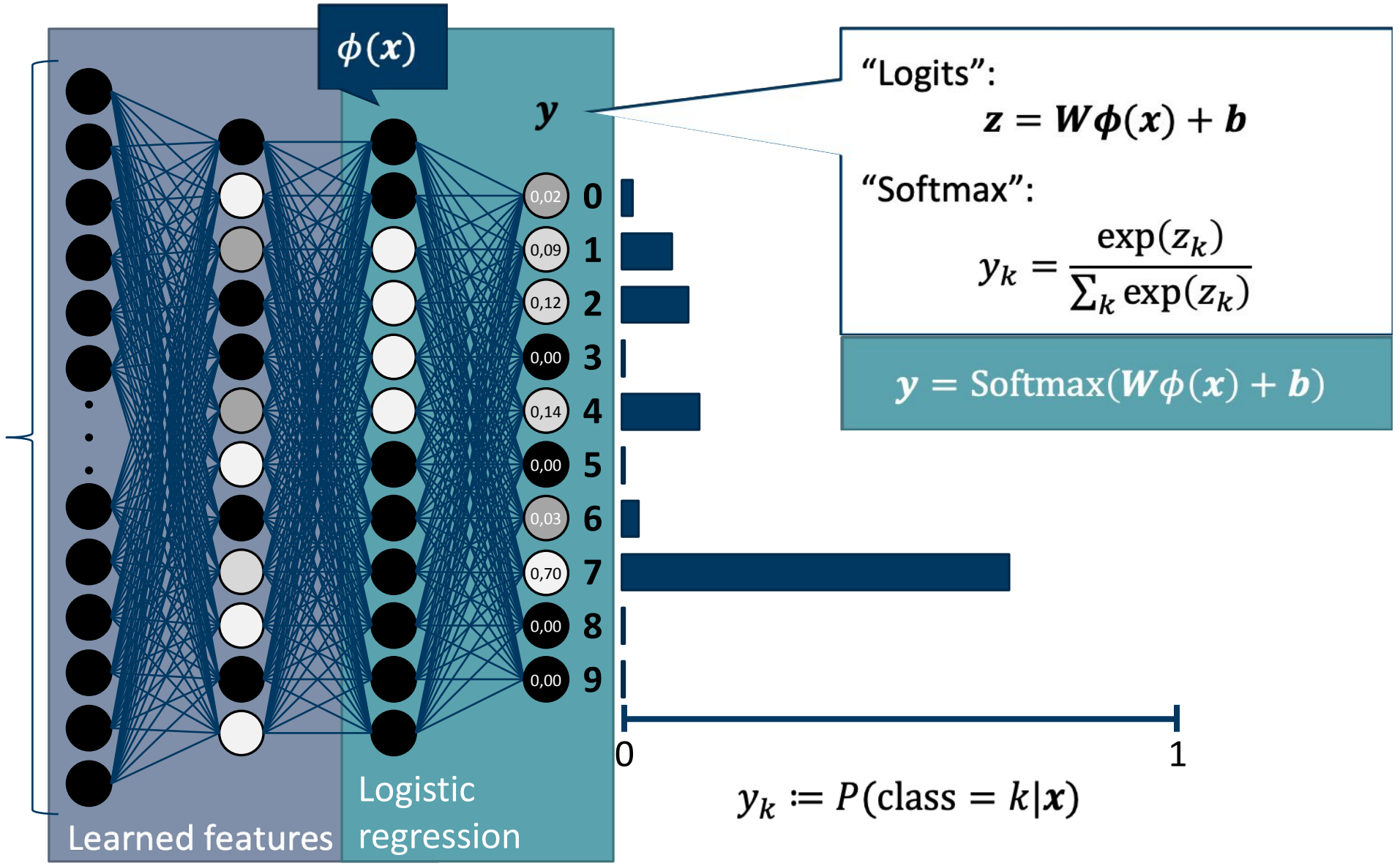
Output layer: probability distribution over classes



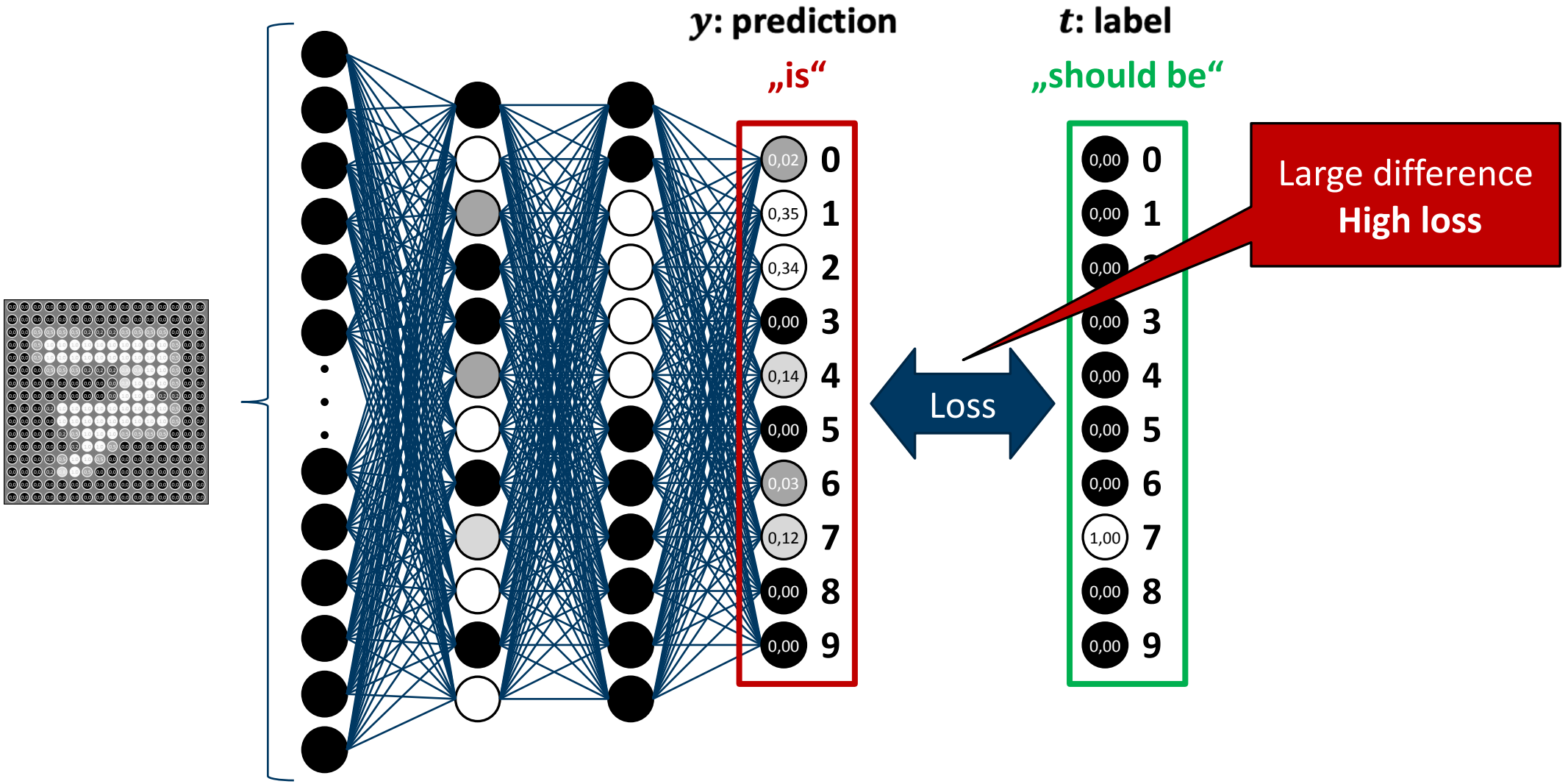


MLP: Logistic regression on learned features

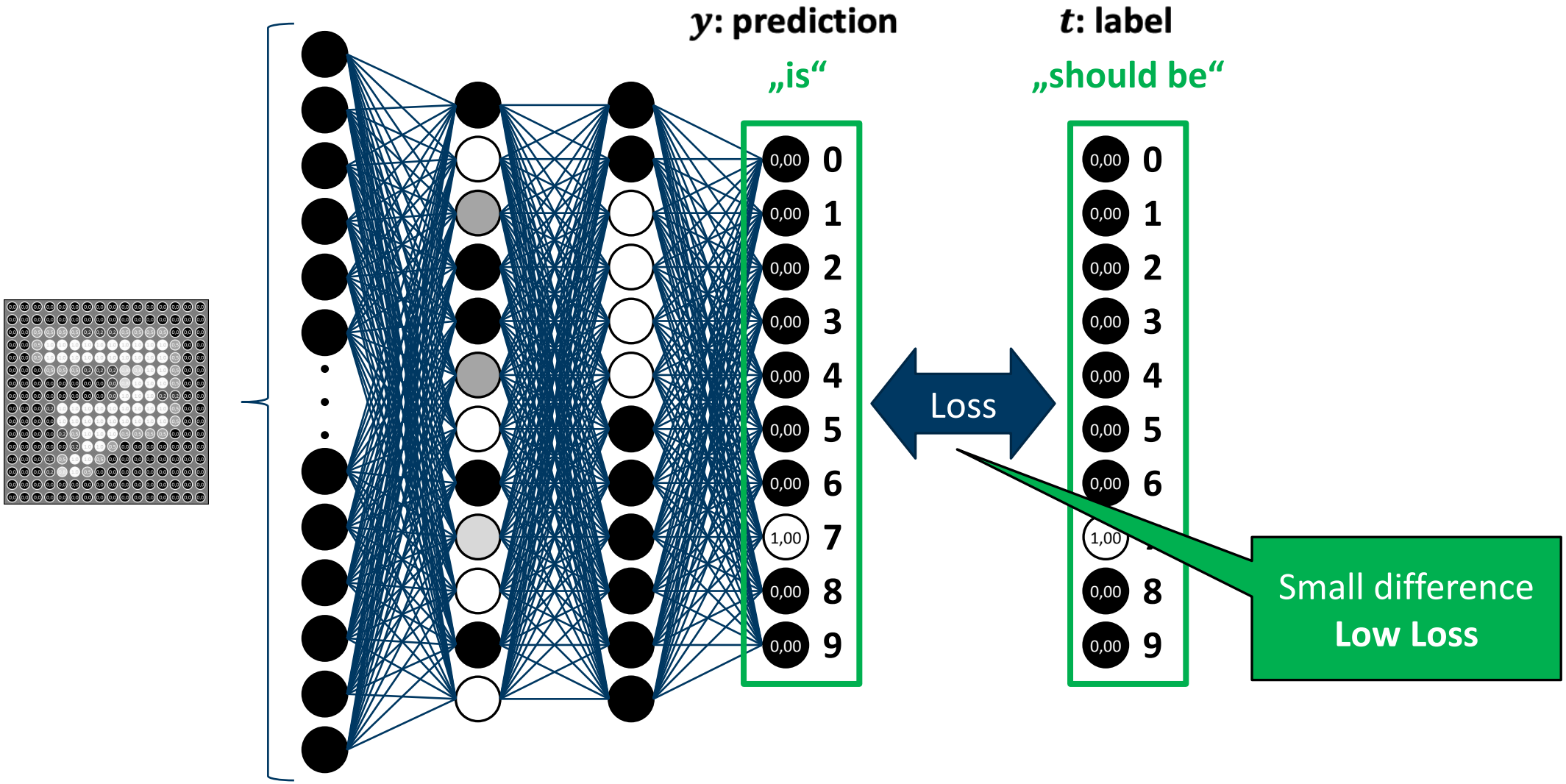




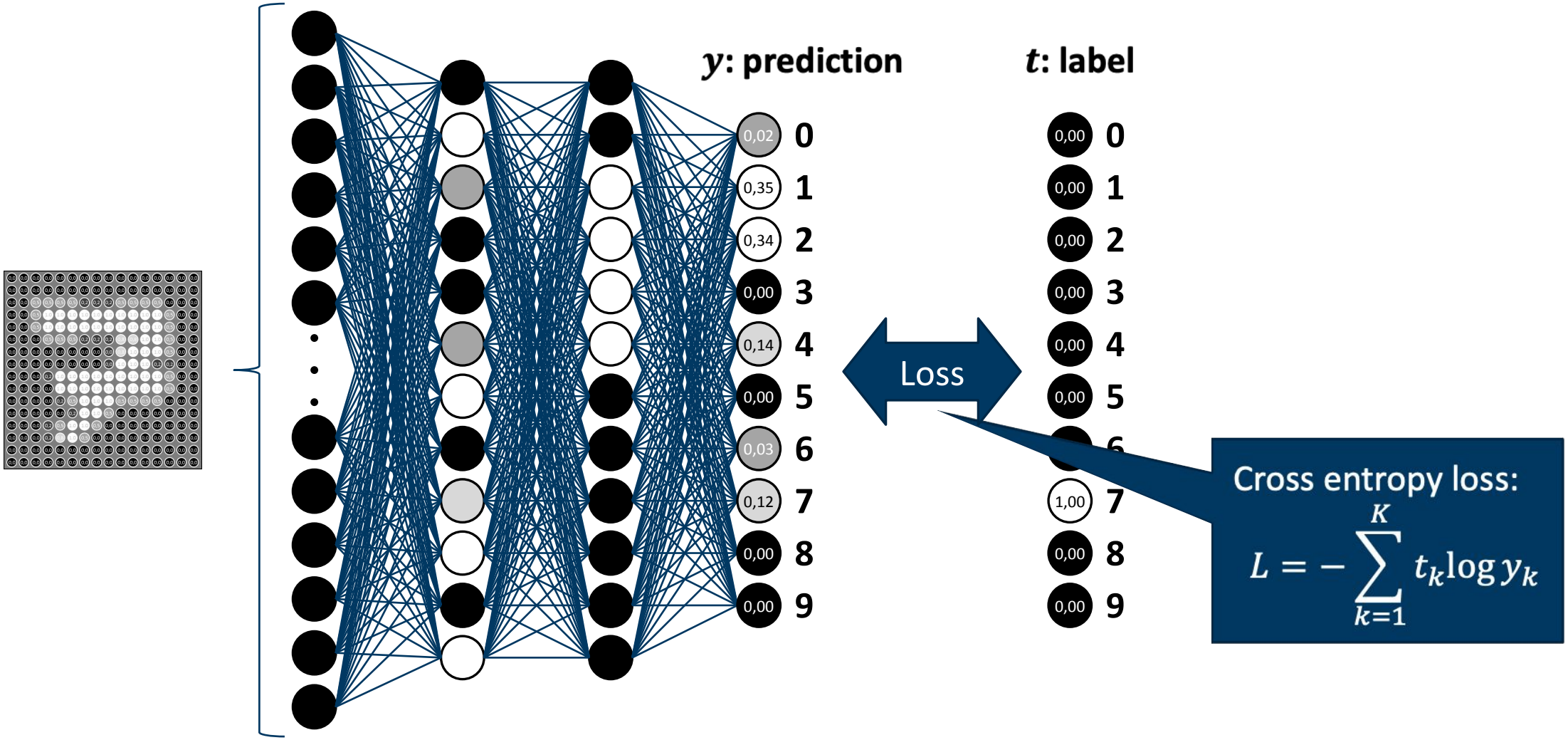
Loss function measures model performance



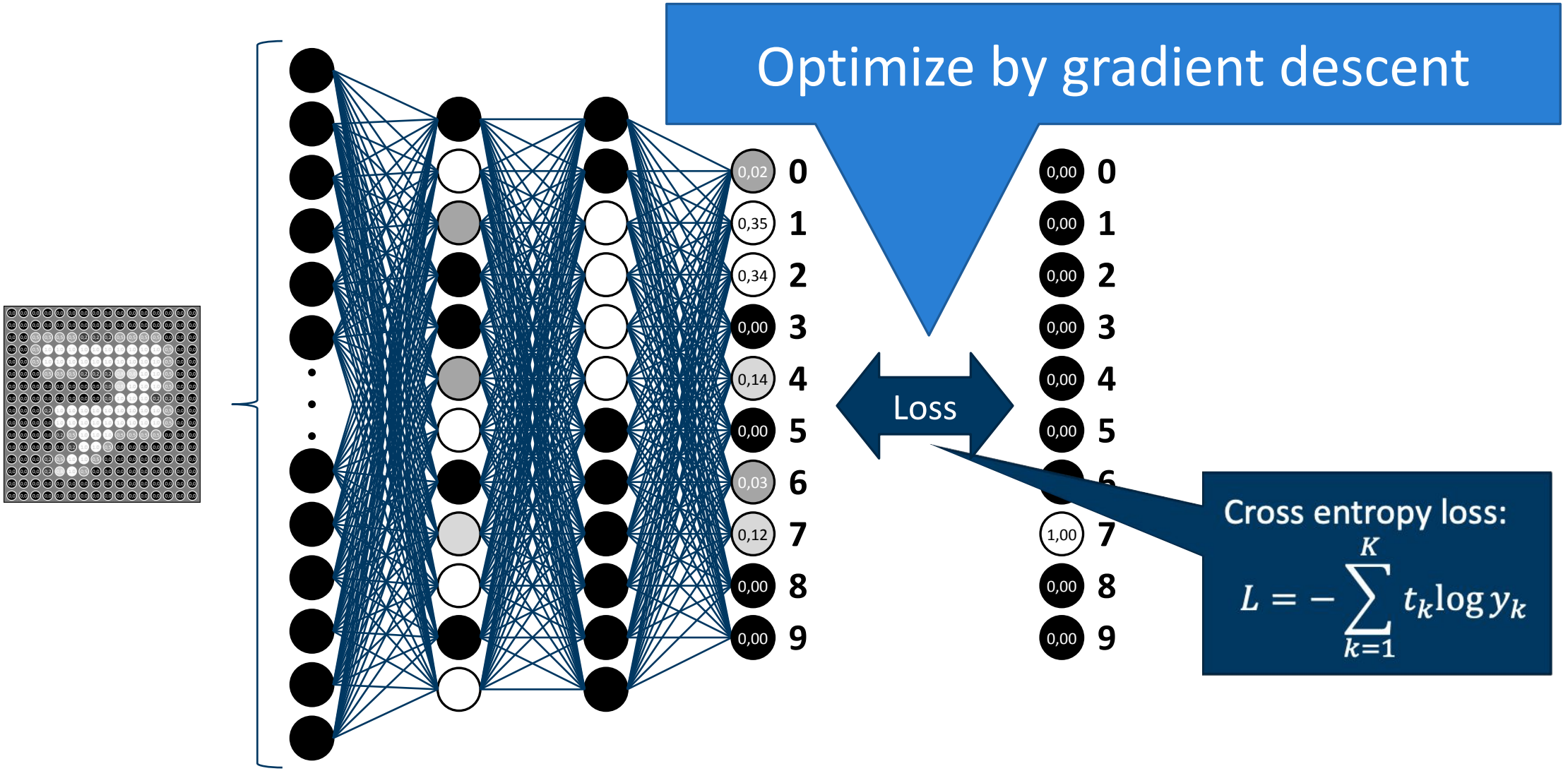
Loss function measures model performance



Loss function for classification: cross entropy



How to optimize the loss function?



Optimize loss function by gradient descent

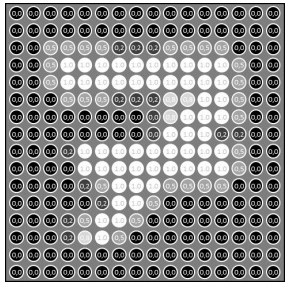
We need:

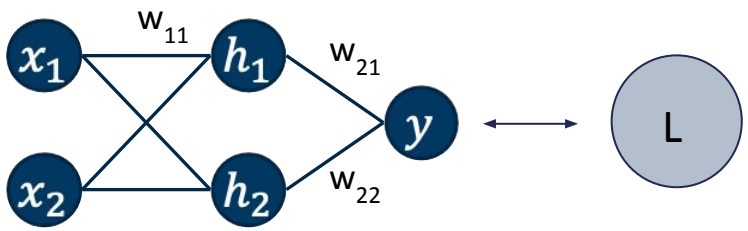
1. Gradient of the loss function w.r.t. the weights, i.e. $\frac{\partial L}{\partial W}$
   -> Backpropagation

2. Algorithm to update the weights
   -> Stochastic gradient descent

Backpropagation

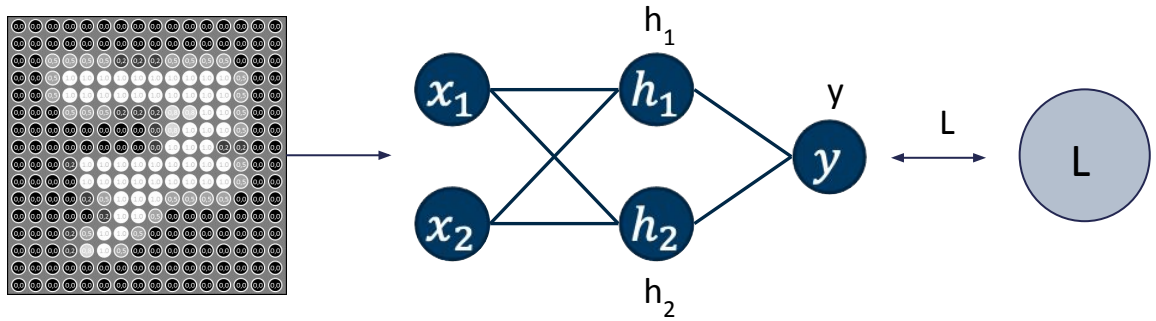
Compute gradients w.r.t. all parameters (weights) in forward and backward pass





Backpropagation - Forward Pass

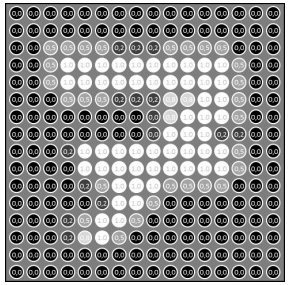
Compute all activations, output and loss and store these values

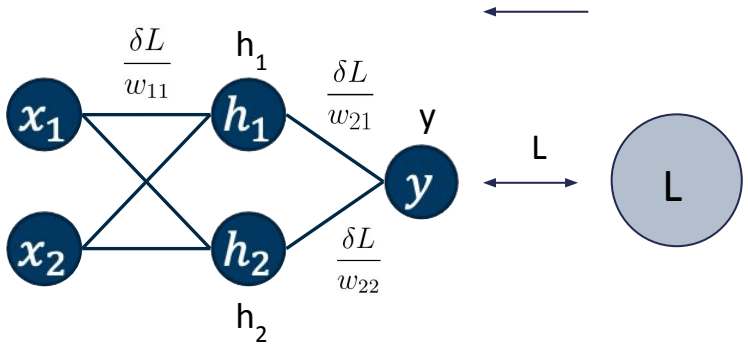


Backpropagation - Backward Pass

Compute gradients w.r.t. the loss for all weights using the chain rule

- Needs activations from forward pass



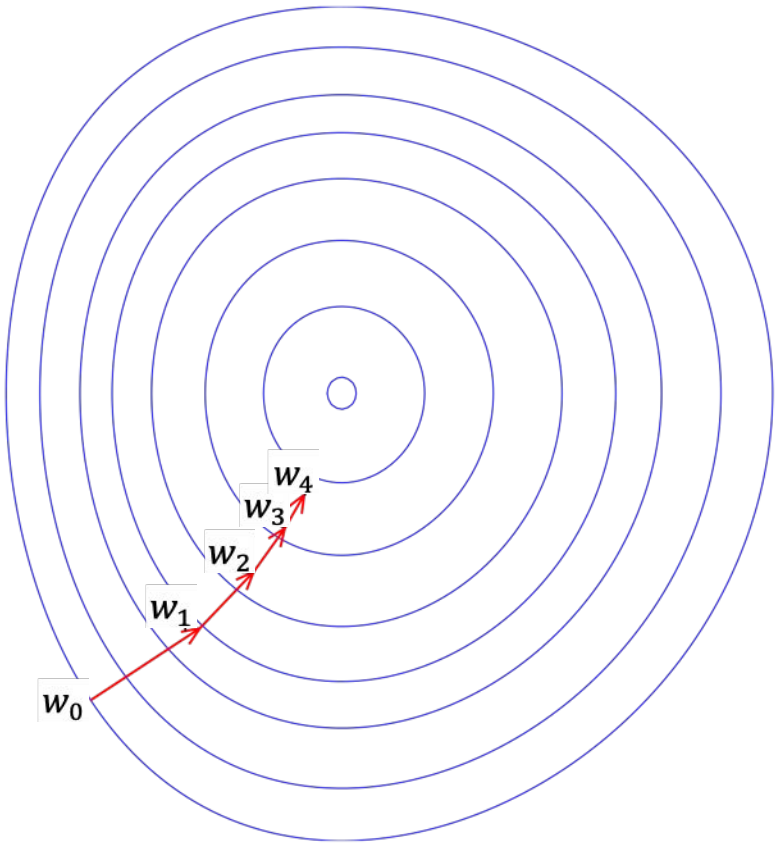


Starting point: gradient descent to minimize the loss function

Start with a random guess for the parameters w

Iteratively update w by a small step λ in the direction of the gradient of the loss function

$$w \leftarrow w - \lambda \nabla_w L(w)$$



Source: Wikipedia

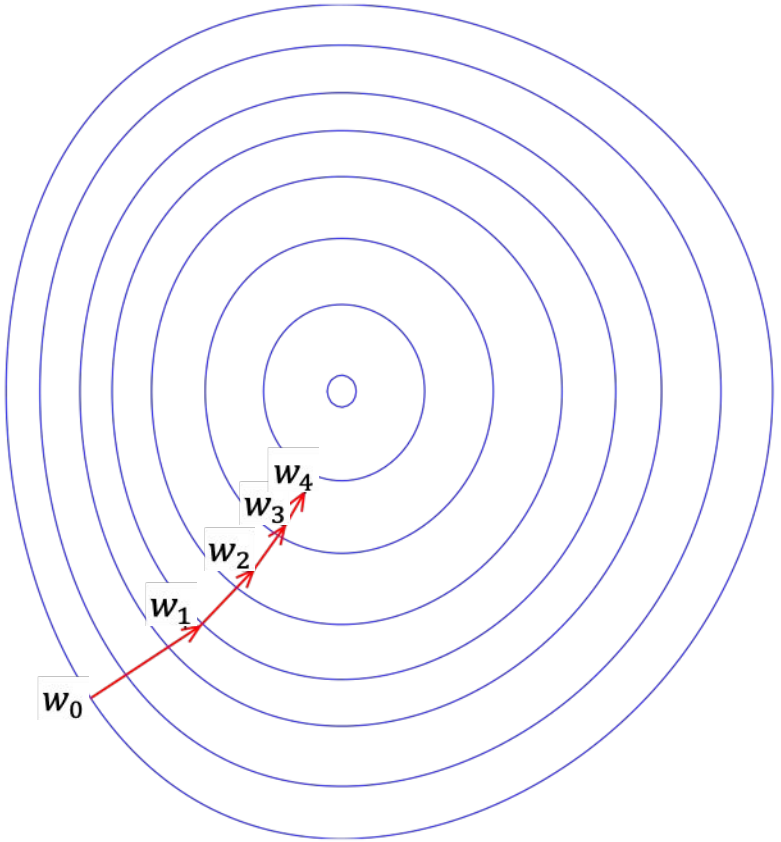
Starting point: gradient descent to minimize the loss function

Start with a random guess for the parameters w

Iteratively update w by a small step λ in the direction of the gradient of the loss function

$$w \leftarrow w - \lambda \nabla_w L(w)$$

Computed via Backprop!



Source: Wikipedia

Automatic differentiation

Each operation „knows“ its gradient w.r.t. its inputs

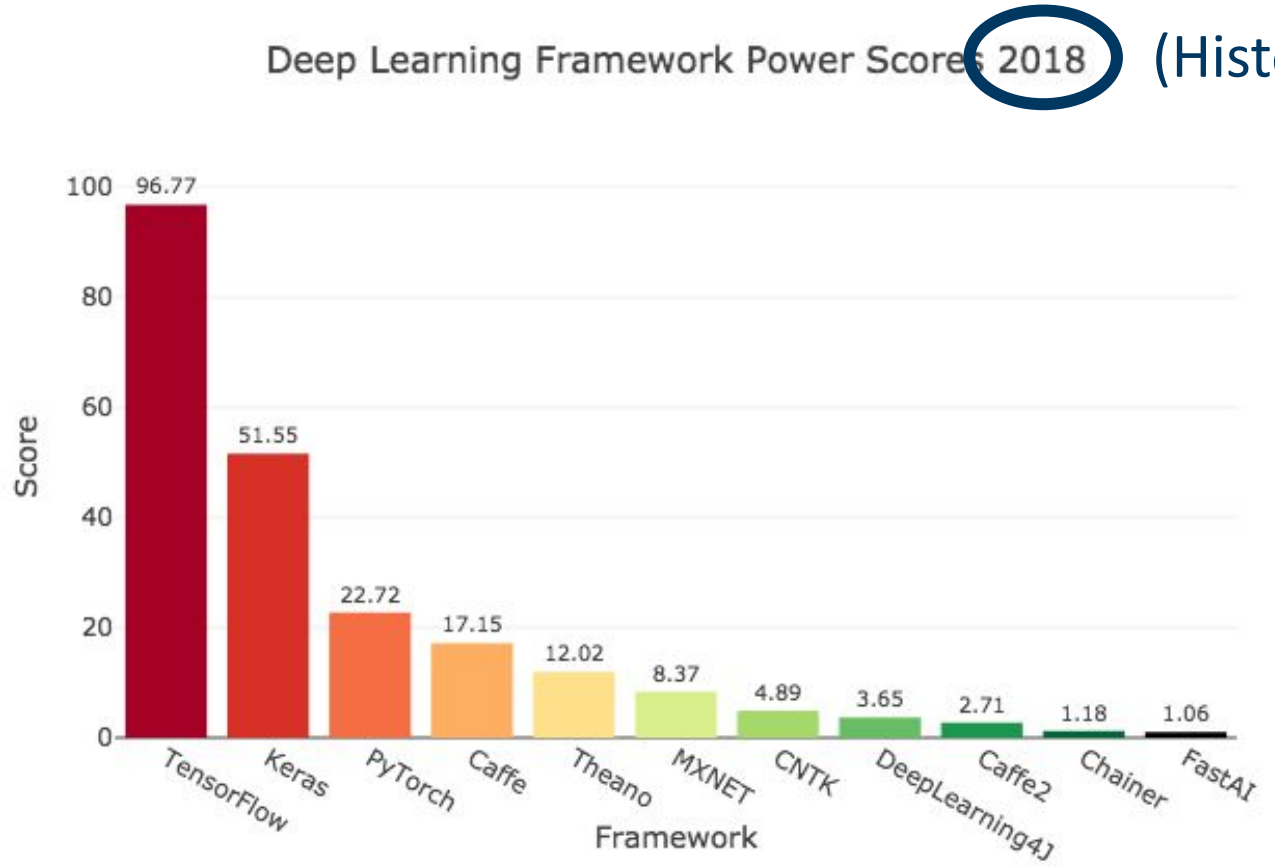
You can compose arbitrary chains of operations (computational graph)

Gradients get computed automatically via chain rule

Modern deep learning frameworks
like *PyTorch, Tensorflow* etc. implement automatic differentiation

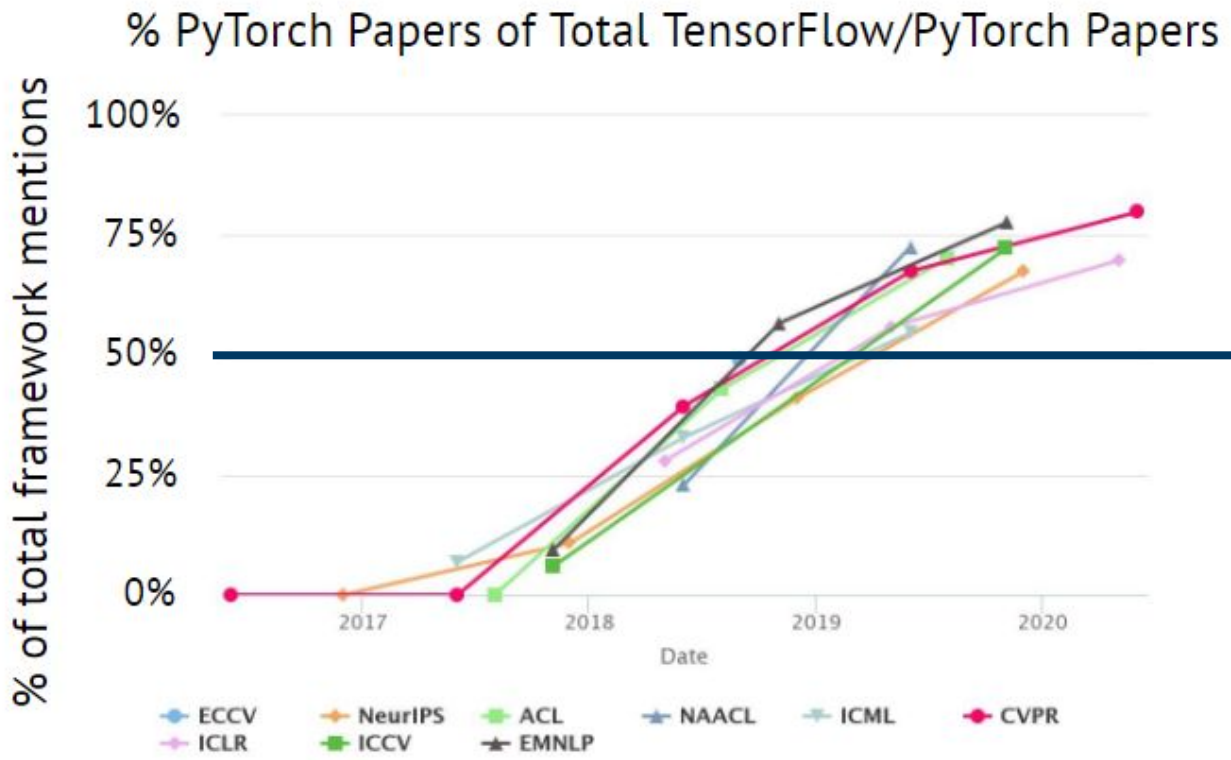
Deep learning frameworks

2018: TensorFlow most widely used framework



(Historical data)

PyTorch surpassed TensorFlow as the most popular framework in 2019



PyTorch > TensorFlow

TensorFlow > PyTorch

https://dlabs.ai/resources/whitepapers/10-machine-learning-frameworks-to-try-in-2021/

What are these frameworks about?

1. **Automatic differentiation (Autodiff)**
2. **Running computations on different hardware backends (e.g. GPUs)**
3. „Layer“ modules
4. Data loading etc.

What are these frameworks about?

1 **Automatic differentiation (Autodiff)**

Each operation „knows“ its gradient w.r.t. its inputs

You can compose arbitrary chains of operations

Gradient gets computed automatically via chain rule

Backpropagation is a special case of “reverse mode” automatic differentiation

Automatic differentiation

PyTorch code

```
x = torch.ones(2, 2,

requires_grad=True)

y = x + 2


z = y * y * 3


o = z.mean()

o.backward()
x.grad
```

Output

```
tensor([[1., 1.],
        [1., 1.]], requires_grad=True)

tensor([[3., 3.],
        [3., 3.]], grad_fn=<AddBackward0>)

tensor([[27., 27.],
        [27., 27.]], grad_fn=<MulBackward0>)

tensor(27., grad_fn=<MeanBackward0>)

tensor([[4.5000, 4.5000],
        [4.5000, 4.5000]])
```

Math

$$x = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$y = x + 2$$

$$z_{ij} = 3y_{ij}^2$$

$$o = \frac{1}{4}\sum_{i,j} z_{ij} = \frac{3}{4}\sum_{i,j}(x_{ij}+2)^2$$

$$\frac{\partial o}{\partial x_{11}} = \frac{3}{2}(x_{11}+2)$$

$$\Rightarrow \left.\frac{\partial o}{\partial x_{11}}\right|_{x_{11}=1} = 4.5$$

What are these frameworks about?

2 **Support for computations on GPU**

GPUs are highly parallel processing units

Originally developed for 3d video gaming

Now mostly used for deep learning and crypto currency mining

Performing computations on GPU with PyTorch

Current success of Deep Learning: advances in hardware development, especially GPUs

GPUs (graphics processing unit): optimized for manipulating graphics, which includes linear algebra tasks like matrix multiplications

- Contrary to CPUs, GPUs perform these calculations in parallel
- Makes them a perfect fit for Deep Learning (given that they have enough memory)
- Companies like NVIDIA today design GPUs intended for Deep Learning

Machine Learning libraries are often designed s.t. if a GPU is available, it can be easily used for computations without having to care too much about internals

Performing computations on GPU with PyTorch

In PyTorch, you can simply move tensors and models to the GPU and back

- If both model and data reside on the GPU, computations will be performed there
- If only one of both is on the GPU, PyTorch will throw an exception

```
# check if a GPU is available
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

data = data.to(device)    # move data tensor to device
model = model.to(device)  # move model to device

output = model(data)  # perform computations on GPU if available

output = output.cpu() # move model to cpu if not already there

output = output.numpy()   # convert tensor to numpy array
```

What are these frameworks about?

3 **Layer modules**

Very fast and easy to build custom deep models

Allow “chaining” different layers together

High-level abstractions for many common concepts and layer types

PyTorch example: nn.Sequential

Models can be easily composed using PyTorch's nn.Sequential() module:

```
model = torch.nn.Sequential(
    torch.nn.Linear(1, 10),
    torch.nn.Dropout(0.5),
    torch.nn.ReLU(),
    torch.nn.Linear(10, 10),
    torch.nn.ReLU(),
    torch.nn.Linear(10, 1),
)

output = model(input)
```

What are these frameworks about?

4 **Data loading and handling**

Data loaders are common abstractions to handle typical workflows

Many tasks regarding training data is recurring, e.g. loading of data, iterating over it, shuffling data, apply augmentation, …

PyTorch helps with providing well designed interfaces and helper classes to handle such tasks

Questions!